

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин
Дисциплина: Базы данных

Тема «ЖД-вокзал»
Лабораторная работа №6
Создание приложения для базы данных

Студент:
Преподаватель:

Д.А. Снитко
Д.В. Куприянова

МИНСК 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 СИСТЕМНЫЕ ТРЕБОВАНИЯ.....	4
1 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	4
1.1 Системные требования.....	4
1.2 Начальный экран.....	4
1.3 Область выбора таблицы.....	5
1.4 Кнопки «Выполнить запрос».....	6
1.5 Кнопка «Добавить строку».....	6
1.6 Кнопка «Удалить строку».....	7
1.7 Кнопка «Экспорт таблицы в Excel».....	8
1.8 Кнопка «Экспорт результата в Excel».....	9
1.9 Кнопка «Новый запрос» и «Сохраненные запросы».....	10
1.10 Кнопка «Создать таблицу».....	13
1.11 Кнопка «Удалить таблицу».....	14
1.12 Кнопка «Резервная копия БД».....	15
1.13 Кнопка «Резервная копия таблицы».....	15
1.14 Кнопка «Восстановление БД».....	15
1.15 Кнопка «Восстановление таблицы».....	16
1.16 Изменение значений полей таблицы.....	17
ЗАКЛЮЧЕНИЕ.....	18
ПРИЛОЖЕНИЕ А.....	19

ВВЕДЕНИЕ

Целью данной работы является разработка прикладного приложения, обеспечивающего взаимодействие пользователя с базой данных через интуитивно понятный интерфейс, без необходимости, но с возможностью написания SQL-запросов вручную.

В рамках задачи предполагается реализовать базовые операции управления данными: создание и удаление таблиц, редактирование их структуры, резервное копирование и восстановление информации.

Разрабатываемое приложение демонстрирует интеграцию PostgreSQL с Python, подчеркивая возможности взаимодействия между клиентской частью и СУБД. Учитывается обеспечение отказоустойчивости через резервное копирование для сохранения целостности данных.

1 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

1.1 Системные требования

Для запуска программы необходимо иметь компьютер со следующими характеристиками:

1. Операционная система: Windows 10 или более поздняя версия.
2. Процессор: не менее 1 ГГц.
3. Место на жестком диске: 512 МБ.
4. ОЗУ: 2 ГБ для 64-разрядной системы.
5. Место на жестком диске: 512 МБ.
6. Видеоадаптер: DirectX 9.
7. Разрешение экрана: минимум 800 x 600.

1.2 Начальный экран

Главное окно приложения представляет собой интерфейс для работы с базой данных. Оно разделено на три основные области: список таблиц, отображение данных и панель для работы с SQL-запросами.

1. Левая панель - содержит список таблиц и элементы управления. Список для выбора таблицы из базы данных. Кнопка "Обновить список" для обновления списка доступных таблиц. Кнопки управления таблицами. "Создать таблицу" – создание новой таблицы. "Удалить таблицу" – удаление текущей таблицы. Кнопки резервного копирования. "Резервная копия БД" – создание резервной копии всей базы данных. "Резервная копия таблицы" – создание резервной копии текущей таблицы. "Восстановить БД" – восстановление базы данных из резервной копии. "Восстановить таблицу" – восстановление таблицы из резервной копии.

2. Центральная область – отображает данные выбранной таблицы в виде интерактивной таблицы. Включает следующие элементы управления. Кнопка "Добавить строку" – добавление новой записи в таблицу. Кнопка "Удалить строку" – удаление выбранной записи. Кнопка "Сохранить изменения" – сохранение внесенных изменений. Кнопки экспорта. "Экспорт таблицы в Excel" – экспорт всей таблицы. "Экспорт результата в Excel" – экспорт результата текущего запроса.

3. Нижняя область – панель для работы с SQL-запросами. Поле ввода SQL-запроса с поддержкой многострочного ввода. Кнопка "Выполнить запрос" – выполнение введенного SQL-запроса. Кнопка "Новый запрос" – сохранение запроса. Кнопка "Сохраненные запросы" – доступ к ранее сохраненным запросам.

Для корректной работы программы необходимо предварительно установить интерпретатор Python и дополнительные библиотеки, указанные в файле зависимостей.

Начальный экран приложения представлен на рисунке 1.1.

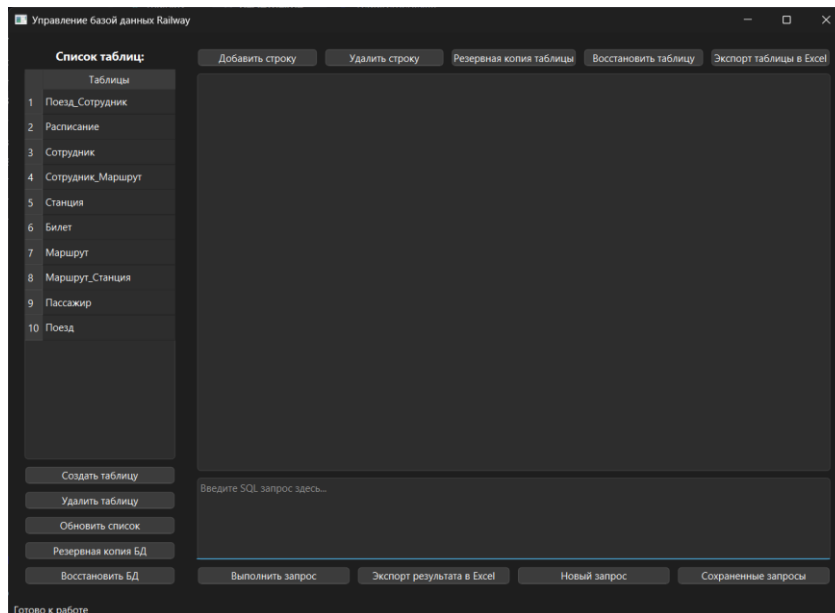


Рисунок 1.1 – Начальный экран приложения

1.3 Область выбора таблицы

Кликнув на любую таблицу из левой панели, содержащей список таблиц, появятся все записи выбранной таблицы.

Выбор таблицы показан на рисунке 1.2.

	id	Пассажир_id	Поезд_id	Маршрут_id	Место	Время_покупки	Стоимость	Категория
1	1	1	3	2	12	2025-02-20 10:30:00	80.5	Эконом
2	2	2	5	4	8	2025-02-18 14:15:00	155.0	Бизнес
3	3	3	7	1	22	2025-02-19 09:00:00	100.0	Эконом
4	4	4	2	5	5	2025-02-17 11:45:00	90.75	Плацкарт
5	5	5	1	3	18	2025-02-16 13:20:00	180.0	Купе
6	6	6	4	7	9	2025-02-21 16:50:00	135.5	Бизнес
7	7	7	6	6	15	2025-02-22 12:00:00	75.0	Эконом
8	8	8	9	2	7	2025-02-23 18:25:00	145.0	Купе
9	9	9	11	4	10	2025-02-20 07:40:00	95.25	Плацкарт
10	10	10	8	1	3	2025-02-19 19:30:00	160.0	Бизнес
11	11	11	13	5	6	2025-02-18 15:00:00	85.0	Эконом
12	12	12	15	3	20	2025-02-21 08:10:00	100.5	Купе
13	13	13	10	2	11	2025-02-17 17:25:00	82.3	Эконом
14	14	14	12	7	14	2025-02-22 09:45:00	130.0	Бизнес
15	15	15	16	6	2	2025-02-20 11:20:00	97.4	Плацкарт

Рисунок 1.2 – Выбор таблицы

1.4 Кнопка «Выполнить запрос»

Для выполнения SQL-запроса необходимо ввести текст запроса в поле ввода в нижней части окна и нажать кнопку «Выполнить запрос». Результат запроса отобразится в центральной области окна в виде таблицы.

Результат выполнения запроса представлен на рисунке 1.3.

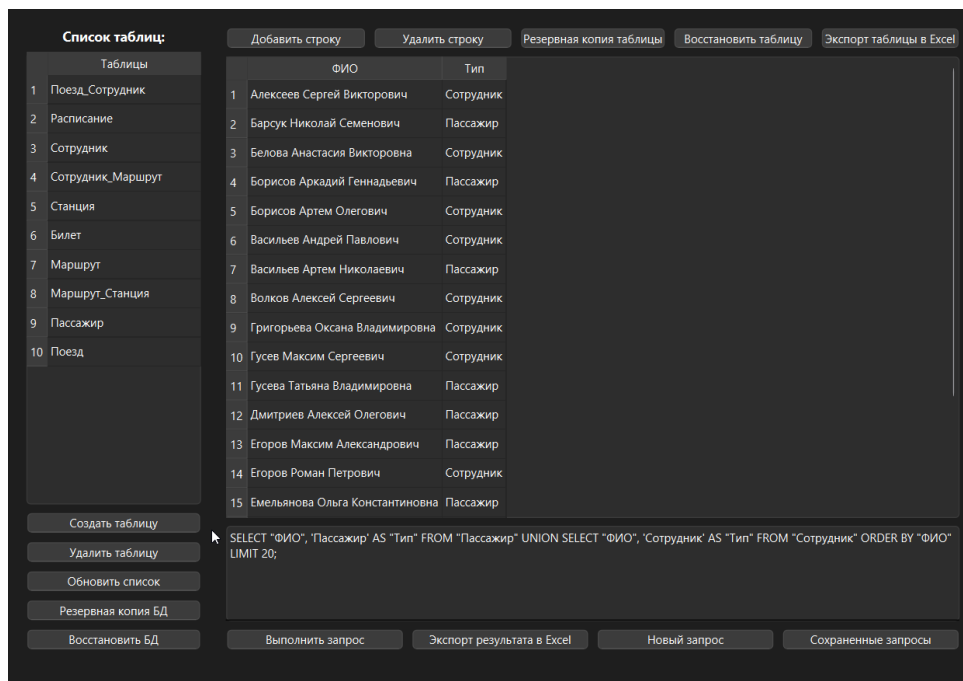


Рисунок 1.3 – Результат запроса

1.5 Кнопка «Добавить строку»

Выбрав таблицу и кликнув по кнопке появляется новое окно, в котором нужно заполнить данные строки таблицы. После нажать кнопку «Сохранить».

На рисунке 1.4 представлена таблица до добавления новой записи (строки), на рисунке 1.5 окно для заполнения полей для добавления записи в базу данных. На рисунке 1.6 представлена таблица с новой записью.

Список таблиц:

Таблицы
1 Поезд_Сотрудник
2 Расписание
3 Сотрудник
4 Сотрудник_Маршрут
5 Станция
6 Билет
7 Маршрут
8 Маршрут_Станция
9 Пассажир
10 Поезд

Добавить строку Удалить строку Резервная копия таблицы Восстановить таблицу Экспорт таблицы в Excel

id	ФИО	Номер_паспорта	Пол	Контактный_телефон
17	Мельникова Юлия Андреевна	BB8234567	Женский	375298889900
18	Тимофеев Игорь Владимирович	BB9234567	Мужской	375299990011
19	Сорокина Надежда Сергеевна	CC1234567	Женский	375291112233
20	Борисов Аркадий Геннадьевич	CC2234567	Мужской	375292223344
21	Никитина Лариса Ивановна	CC3234567	Женский	375293334455
22	Савельев Дмитрий Викторович	CC4234567	Мужской	375294445566
23	Фролова Оксана Владимировна	CC5234567	Женский	375295556677
24	Рябов Евгений Анатольевич	CC6234567	Мужской	375296667788
25	Полякова Мария Олеговна	CC7234567	Женский	375297778899
26	Щербаков Алексей Павлович	CC8234567	Мужской	375298889900
27	Шевченко Галина Андреевна	CC9234567	Женский	375299990011
28	Федоров Николай Семенович	DD1234567	Мужской	375291112244
29	Емельянова Ольга Константиновна	DD2239567	Женский	375299997011
30	Сорокина Галина Андреевна	CC9664566	Женский	375288990011
31	Барсук Николай Семенович	DD1114567	Мужской	375295512244

Создать таблицу Удалить таблицу Обновить список Резервная копия БД Восстановить БД

```
SELECT "ФИО", 'Пассажир' AS "Тип" FROM "Пассажир" UNION SELECT "ФИО", 'Сотрудник' AS "Тип" FROM "Сотрудник" ORDER BY "ФИО" LIMIT 20;
```

Выполнить запрос Экспорт результата в Excel Новый запрос Сохраненные запросы

Рисунок 1.4 – Таблица Пассажир до добавления новой записи

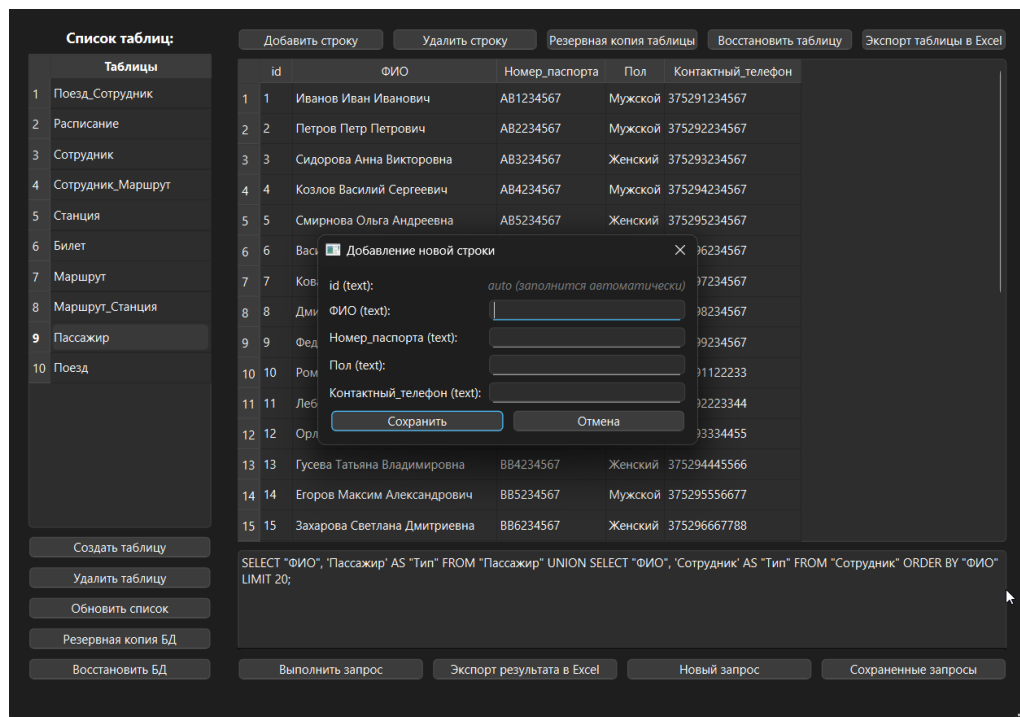


Рисунок 1.5 – Окно добавления новой записи

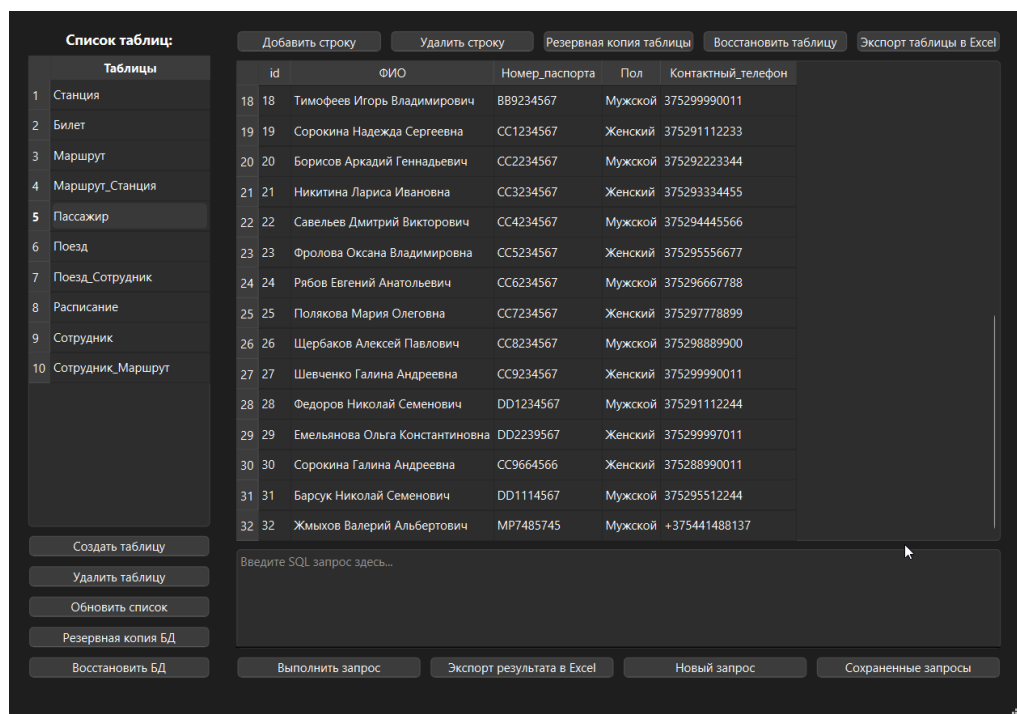


Рисунок 1.6 – Измененная таблица Пассажир с новой записью

1.6 Кнопка «Удалить строку»

Для того, чтобы удалить запись (строку) из таблицы необходимо выделить ее, кликнув по ней, а после кликнуть на кнопку «Удалить строку». После кликнуть на кнопку «Yes» в окне подтверждения действия.

На рисунке 1.7 представлено окно подтверждения удаления записи. На рисунке 1.8 представлена измененная таблица.

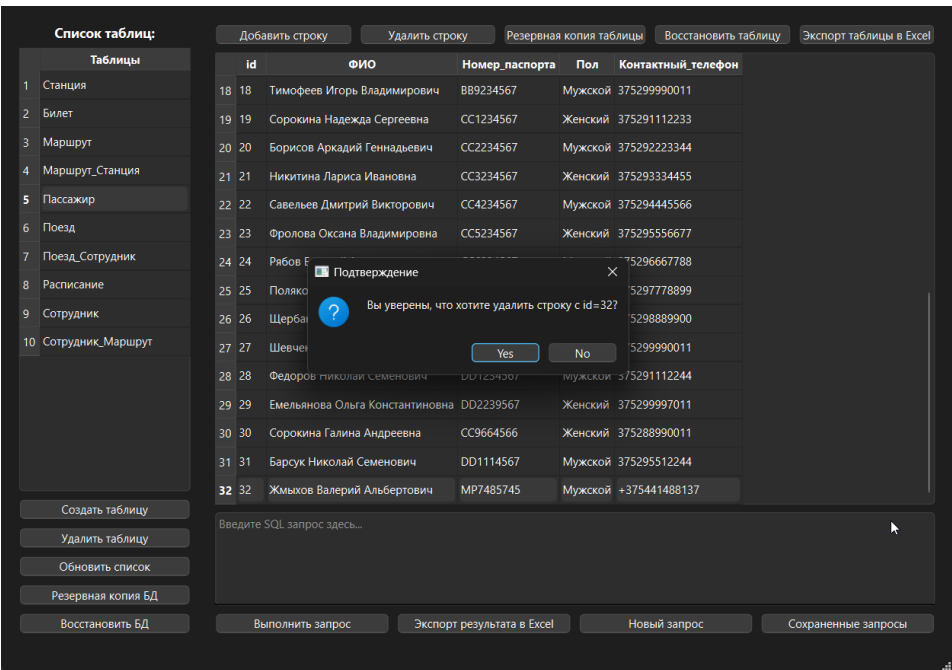


Рисунок 1.7 – Окно подтверждения удаления записи

Список таблиц:

Таблицы
1 Станция
2 Билет
3 Маршрут
4 Маршрут_Станция
5 Пассажир
6 Поезд
7 Поезд_Сотрудник
8 Расписание
9 Сотрудник
10 Сотрудник_Маршрут

Добавить строку Удалить строку Резервная копия таблицы Восстановить таблицу Экспорт таблицы в Excel

id	ФИО	Номер_паспорта	Пол	Контактный_телефон
17	Мельникова Юлия Андреевна	BB8234567	Женский	375298889900
18	Тимофеев Игорь Владимирович	BB9234567	Мужской	375299990011
19	Сорокина Надежда Сергеевна	CC1234567	Женский	375291112233
20	Борисов Аркадий Геннадьевич	CC2234567	Мужской	375292223344
21	Никитина Лариса Ивановна	CC3234567	Женский	375293334455
22	Савельев Дмитрий Викторович	CC4234567	Мужской	375294445566
23	Фролова Оксана Владимировна	CC5234567	Женский	375295556677
24	Рябов Евгений Анатольевич	CC6234567	Мужской	375296667788
25	Полякова Мария Олеговна	CC7234567	Женский	375297778899
26	Щербаков Алексей Павлович	CC8234567	Мужской	375298889900
27	Шевченко Галина Андреевна	CC9234567	Женский	375299990011
28	Федоров Николай Семенович	DD1234567	Мужской	375291112244
29	Емельянова Ольга Константиновна	DD2239567	Женский	375299997011
30	Сорокина Галина Андреевна	CC9664566	Женский	375288990011
31	Барук Николай Семенович	DD1114567	Мужской	375295512244

Создать таблицу Удалить таблицу Обновить список Резервная копия БД Восстановить БД

Введите SQL запрос здесь...

Выполнить запрос Экспорт результата в Excel Новый запрос Сохраненные запросы

Рисунок 1.8 – Измененная таблица Пассажир

1.7 Кнопка «Экспорт таблицы в Excel»

Для того, чтобы экспортировать текущую страницу с записями в виде «Excel» файла необходимо кликнуть кнопку «Экспорт таблицы в Excel».

После выбрать место на диске, куда будет сохранена таблица, задать имя файлу и кликнуть «Сохранить». По указанному пути будет создан файл типа «xlsx».

На рисунке 1.9 представлена текущая страница в excel файле.

id	ФИО	Номер_паспорта	Пол	Контактный_телефон
1	Иванов Иван Иванович	AB1234567	Мужской	375291234567
2	Петров Петр Петрович	AB2234567	Мужской	375292234567
3	Сидорова Анна Викторовна	AB3234567	Женский	375293234567
4	Козлов Василий Сергеевич	AB4234567	Мужской	375294234567
5	Смирнова Ольга Андреевна	AB5234567	Женский	375295234567
6	Васильев Артем Николаевич	AB6234567	Мужской	375296234567
7	Коваленко Дарья Алексеевна	AB7234567	Женский	375297234567
8	Дмитриев Алексей Олегович	AB8234567	Мужской	375298234567
9	Федорова Марина Павловна	AB9234567	Женский	375299234567
10	Романов Сергей Викторович	BB1234567	Мужской	375291122233
11	Лебедева Екатерина Сергеевна	BB2234567	Женский	375292223344
12	Орлов Николай Иванович	BB3234567	Мужской	375293334455
13	Гусева Татьяна Владимировна	BB4234567	Женский	375294445566
14	Егоров Максим Александрович	BB5234567	Мужской	375295556677
15	Захарова Светлана Дмитриевна	BB6234567	Женский	375296667788
16	Кириллов Павел Станиславович	BB7234567	Мужской	375297778899
17	Мельникова Юлия Андреевна	BB8234567	Женский	375298889900
18	Тимофеев Игорь Владимирович	BB9234567	Мужской	375299990011
19	Сорокина Надежда Сергеевна	CC1234567	Женский	375291112233
20	Борисов Аркадий Геннадьевич	CC2234567	Мужской	375292223344
21	Никитина Лариса Ивановна	CC3234567	Женский	375293334455
22	Савельев Дмитрий Викторович	CC4234567	Мужской	375294445566
23	Фролова Оксана Владимировна	CC5234567	Женский	375295556677
24	Рябов Евгений Анатольевич	CC6234567	Мужской	375296667788
25	Полякова Мария Олеговна	CC7234567	Женский	375297778899
26	Щербаков Алексей Павлович	CC8234567	Мужской	375298889900
27	Шевченко Галина Андреевна	CC9234567	Женский	375299990011
28	Федоров Николай Семенович	DD1234567	Мужской	375291112244
29	Емельянова Ольга Константиновна	DD2239567	Женский	375299997011
30	Сорокина Галина Андреевна	CC9664566	Женский	375288990011
31	Барсук Николай Семенович	DD1114567	Мужской	375295512244

Рисунок 1.9 – Текущая страница Пассажир в excel файле

1.8 Кнопка «Экспорт результата в Excel»

Для того, чтобы экспортировать результат выполнения SQL-запроса в виде «excel» файла необходимо кликнуть кнопку «Экспорт результата в Excel». После выбрать место на диске, куда будет сохранена таблица, задать имя файлу и кликнуть «Сохранить». По указанному пути будет создан файл типа «xlsx».

На рисунке 1.10 приведено окно с выбором сохраненного запроса «Билеты дороже 100», который отображает билеты со стоимостью больше 100, а на рисунке 1.11 представлен результат выполнения SQL-запроса в excel файле.

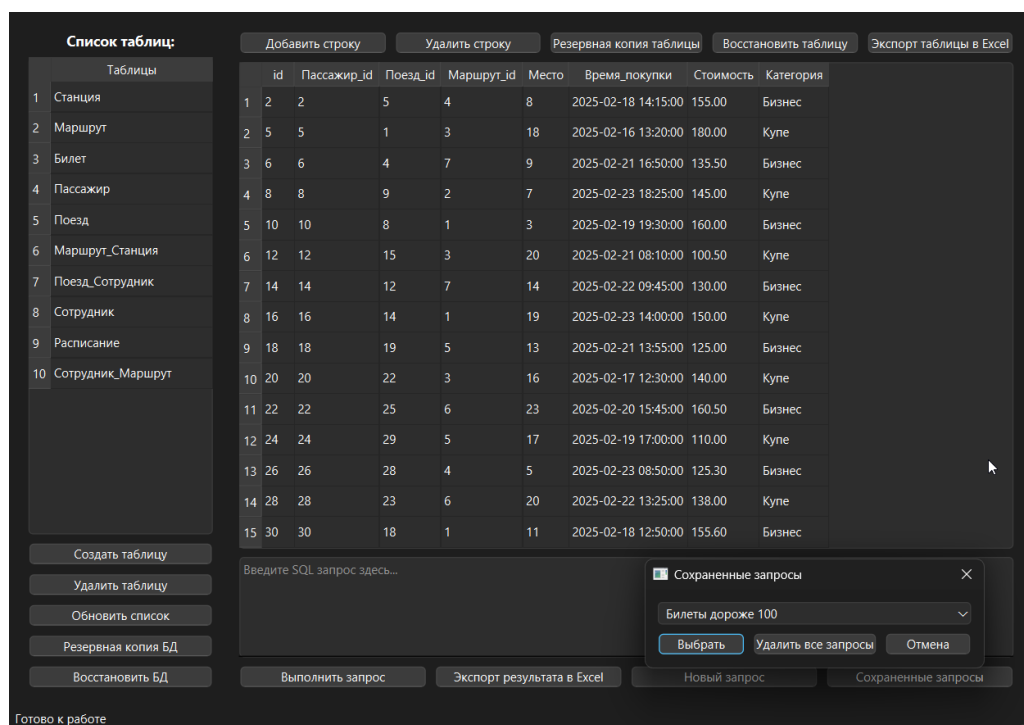


Рисунок 1.10 – Окно с выбором сохраненного запроса «Билеты дороже 100»

id	Пассажир_id	Поезд_id	Маршрут_id	Место	Время_покупки	Стоимость	Категория
2	2	5	4	8	2025-02-18 14:15:00	155.00	Бизнес
5	5	1	3	18	2025-02-16 13:20:00	180.00	Купе
6	6	4	7	9	2025-02-21 16:50:00	135.50	Бизнес
8	8	9	2	7	2025-02-23 18:25:00	145.00	Купе
10	10	8	1	3	2025-02-19 19:30:00	160.00	Бизнес
12	12	15	3	20	2025-02-21 08:10:00	100.50	Купе
14	14	12	7	14	2025-02-22 09:45:00	130.00	Бизнес
16	16	14	1	19	2025-02-23 14:00:00	150.00	Купе
18	18	19	5	13	2025-02-21 13:55:00	125.00	Бизнес
20	20	22	3	16	2025-02-17 12:30:00	140.00	Купе
22	22	25	6	23	2025-02-20 15:45:00	160.50	Бизнес
24	24	29	5	17	2025-02-19 17:00:00	110.00	Купе
26	26	28	4	5	2025-02-23 08:50:00	125.30	Бизнес
28	28	23	6	20	2025-02-22 13:25:00	138.00	Купе
30	30	18	1	11	2025-02-18 12:50:00	155.60	Бизнес

Рисунок 1.11 – Результаты запроса в excel файле

1.9 Кнопка «Новый запрос» и «Сохраненные запросы»

Кликнув на кнопку «Новый запрос» открывается окно, в котором можно задать название запроса и сам SQL-запрос для сохранения и дальнейшего быстрого использования (рисунок 1.12). А также можно загрузить один или несколько запросов из файла. После сохранения в выпадающем списке «Сохраненных запросов» появится этот запрос и его можно будет выполнить (рисунок 1.13).

При нажатии на «Сохраненные запросы», в выпадающем списке можно выбрать заранее сохраненные запросы из лабораторных работ номер 4 и 5 (рисунок 1.14). Выбрав из выпадающего списка определенный запрос (например «Поезда и маршруты») и кликнув по кнопке «Выбрать» запрос исполнится и в центральной области будет выведен результат запроса (рисунок 1.15).

Чтобы удалить сохраненные запросы нужно кликнуть по кнопке «Удалить все запросы» и в новом окне подтвердить удаление, нажав на кнопку «Yes» (рисунок 1.16).

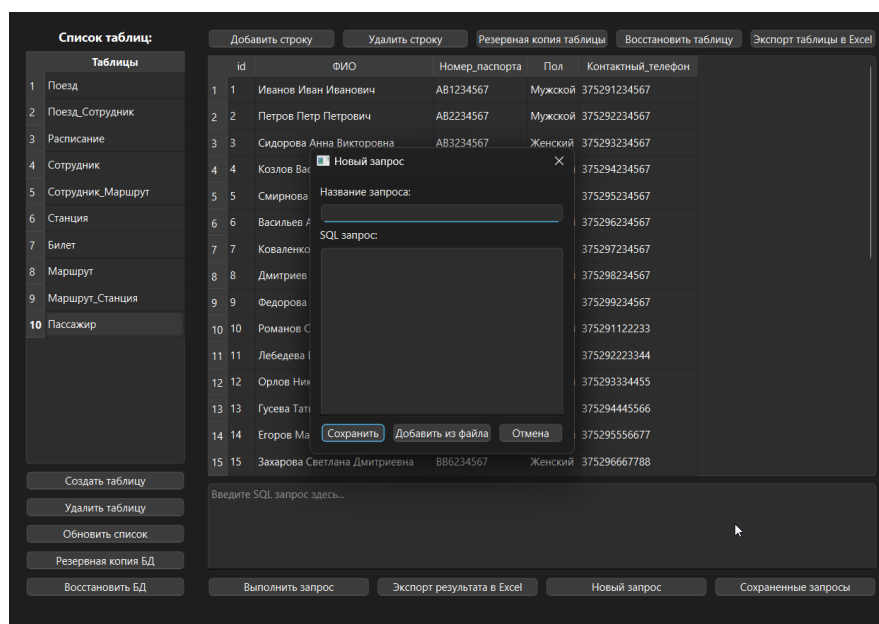


Рисунок 1.12 – Окно сохранения SQL-запроса

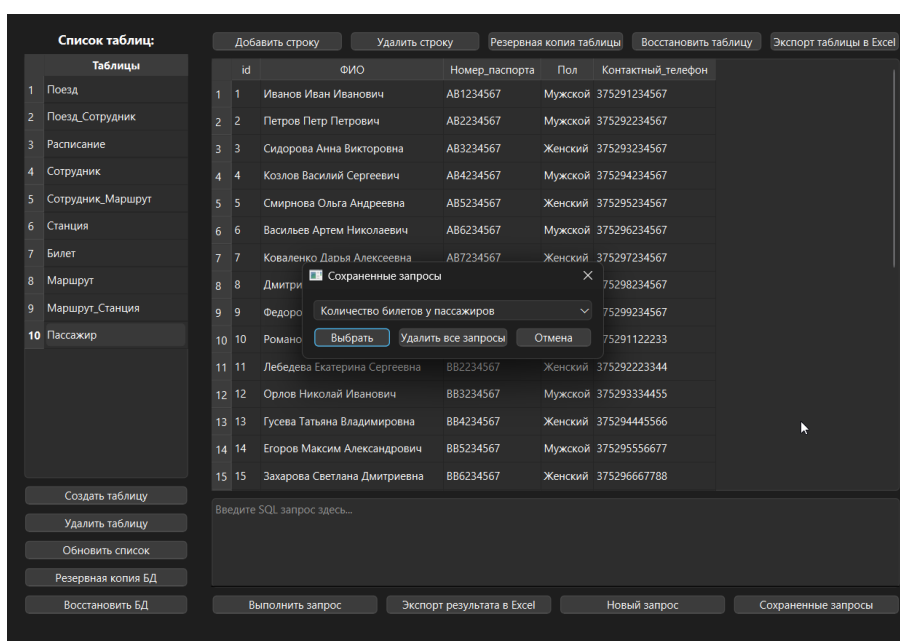


Рисунок 1.13 – Добавленный запрос в списке сохраненных запросов

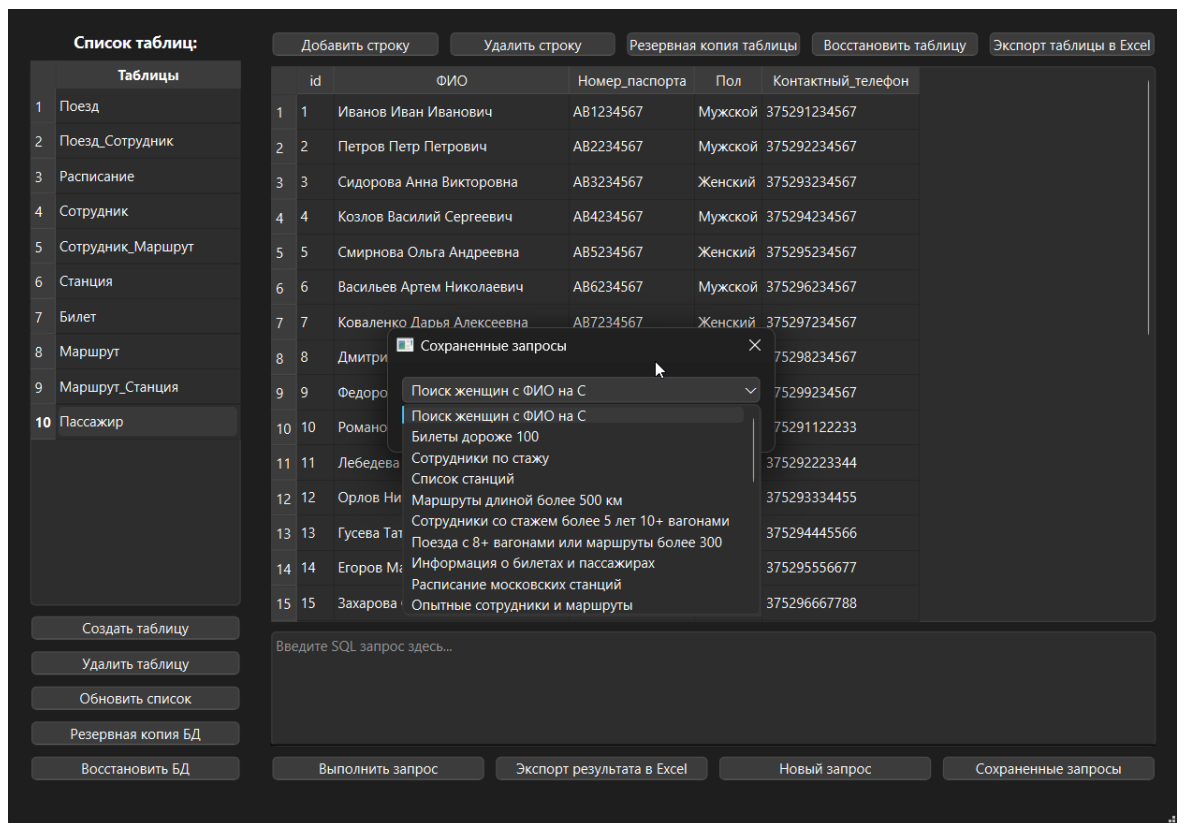


Рисунок 1.14 – Выпадающий список сохраненных запросов

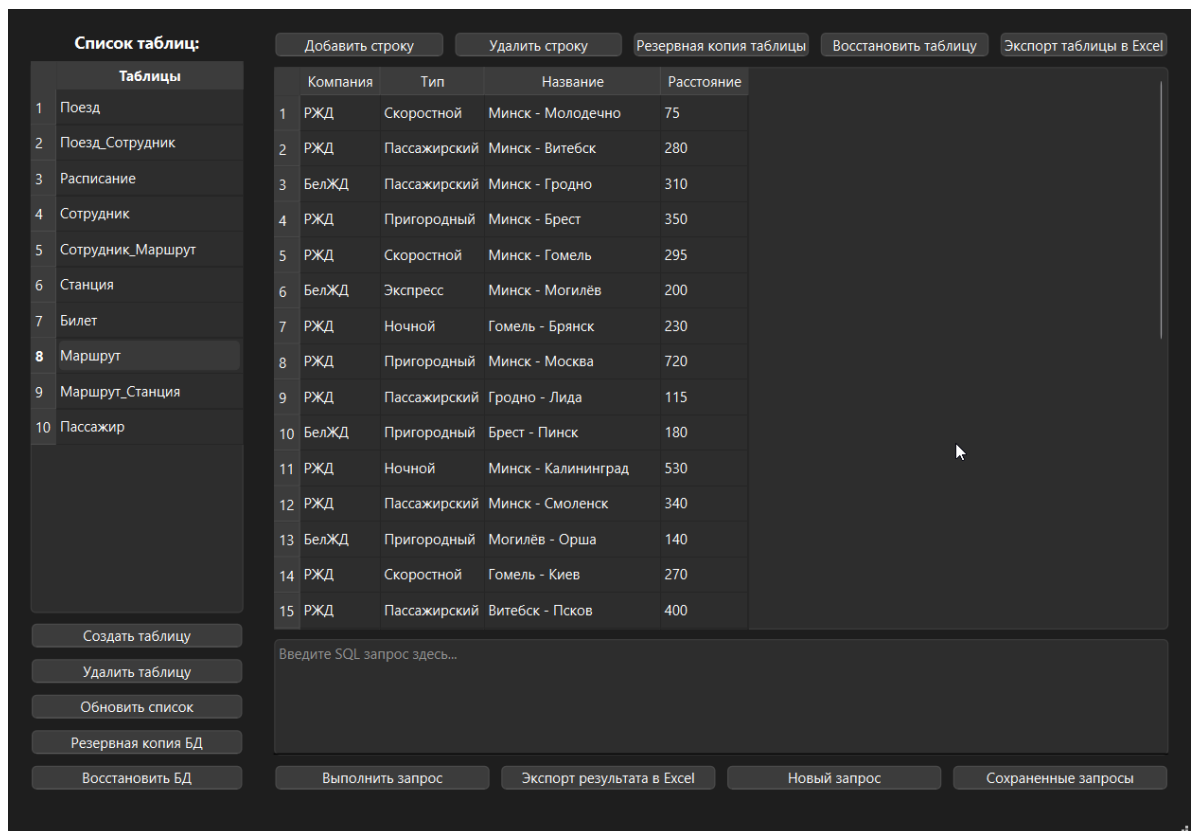


Рисунок 1.15 – Результат выполнения сохраненного запроса «Поезда и маршруты»

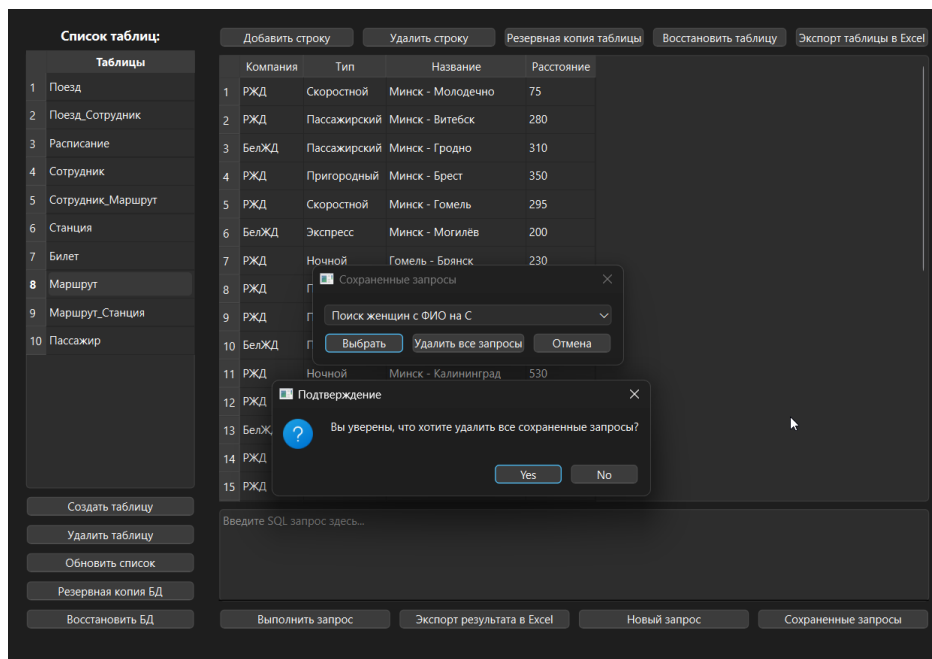


Рисунок 1.16 – Подтверждение удаления всех сохраненных запросов

1.10 Кнопка «Создать таблицу»

Чтобы создать таблицу, необходимо кликнуть на кнопку «Создать таблицу». В новом окне необходимо ввести имя таблицы (рисунок 1.17), после, ввести имя столбца (рисунок 1.18), нажать кнопку «ОК», и ввести тип данных из выпадающего списка (рисунок 1.19), либо оставить поле ввода имени столбца пустым для создания таблицы и нажать кнопку «ОК».

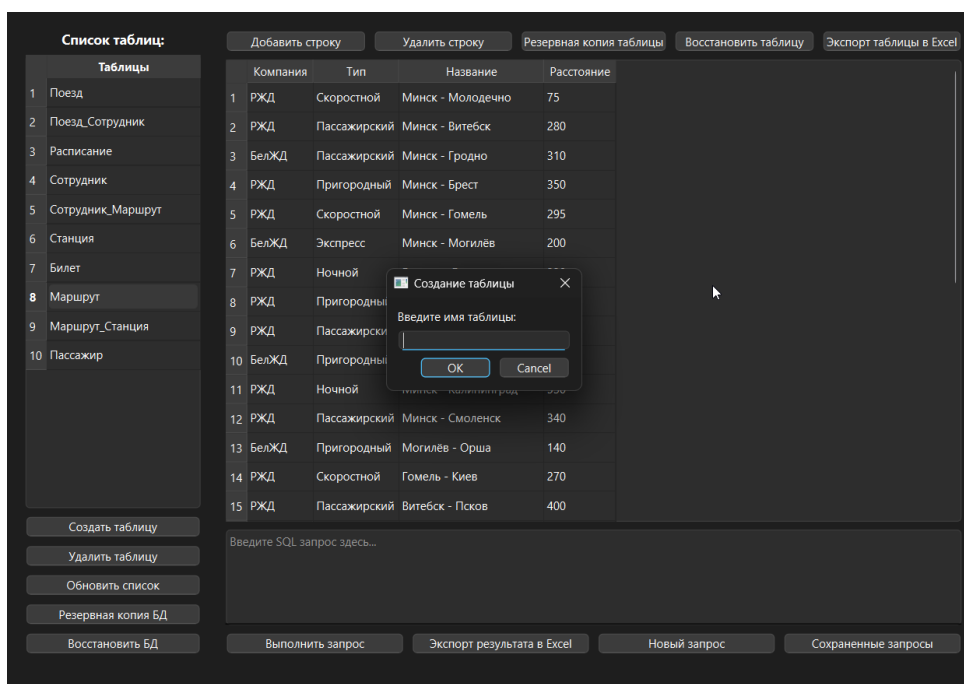


Рисунок 1.17 – Ввод имени новой таблицы

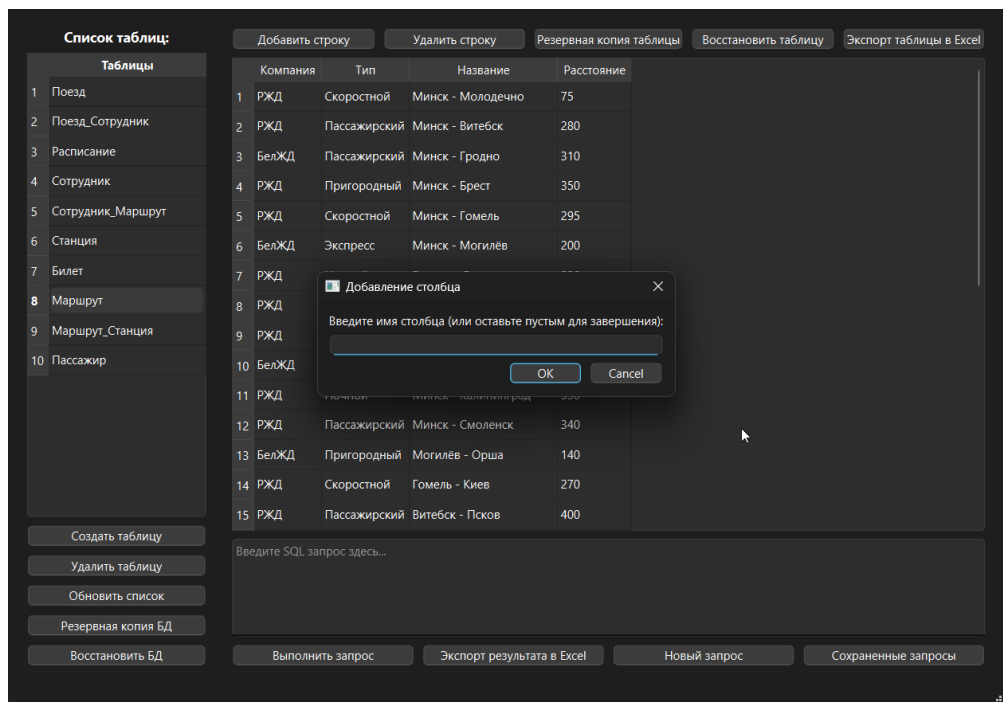


Рисунок 1.18 – Ввод имени столбца новой таблицы

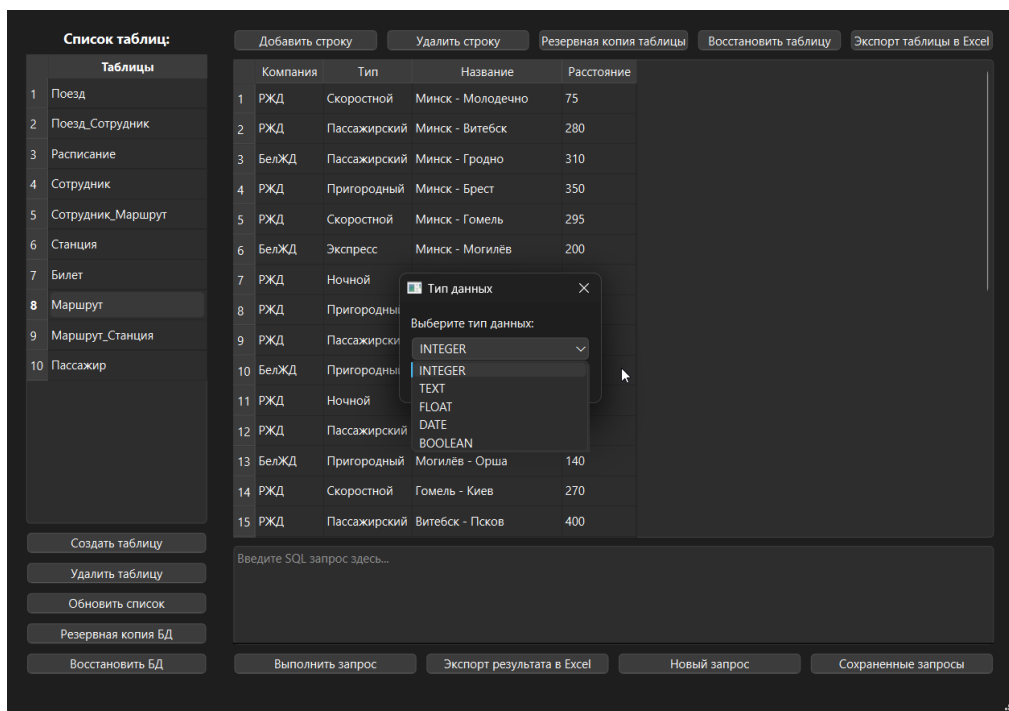


Рисунок 1.19 – Выбор типа данных новой таблицы

1.11 Кнопка «Удалить таблицу»

Чтобы удалить таблицу необходимо выбрать таблицу, нажатием на нее, а после кликнуть по кнопке «Удалить таблицу». В новом окне подтвердить удаление кликнув по кнопке «Yes» (рисунок 1.20).

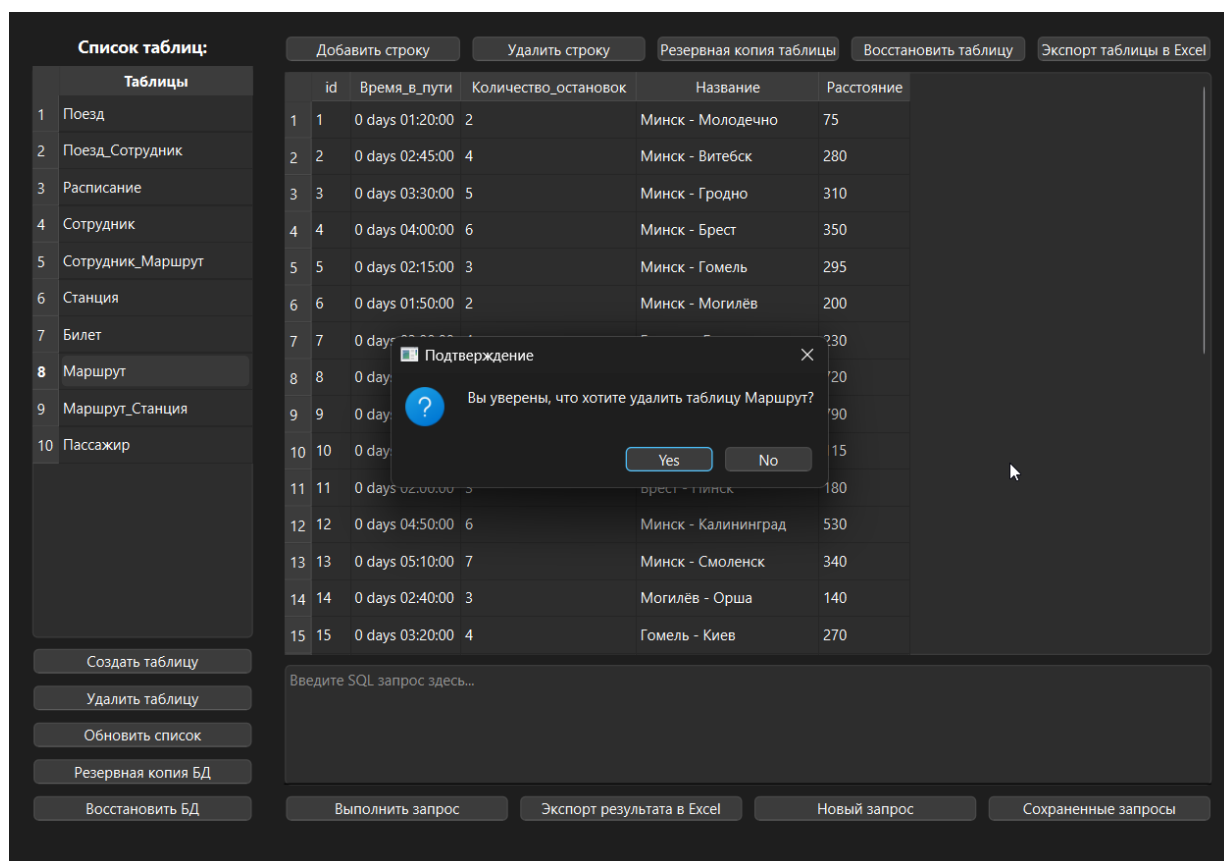


Рисунок 1.20 – Подтверждение удаления таблицы

1.12 Кнопка «Резервная копия БД»

Для того, чтобы сохранить резервную копию базы данных нужно нажать кнопку «Резервная копия БД». После выбрать место на диске, куда будет сохранена таблица, задать имя файлу и кликнуть «Сохранить». По указанному пути будет создана папка с файлами для восстановления базы данных.

1.13 Кнопка «Резервная копия таблицы»

Для того, чтобы сохранить резервную копию таблицы нужно выбрать таблицу и нажать кнопку «Резервная копия таблицы». После выбрать место на диске, куда будет сохранена таблица, задать имя файлу и кликнуть «Сохранить». По указанному пути будет создан файл для восстановления таблицы.

1.14 Кнопка «Восстановить БД»

Для того, чтобы восстановить базу данных из резервной копии необходимо кликнуть по кнопке «Восстановить БД». В новом окне выбрать путь к папке с резервной копией базы данных. Подтвердить восстановление, кликнув по кнопке «Yes» (рисунок 1.21).

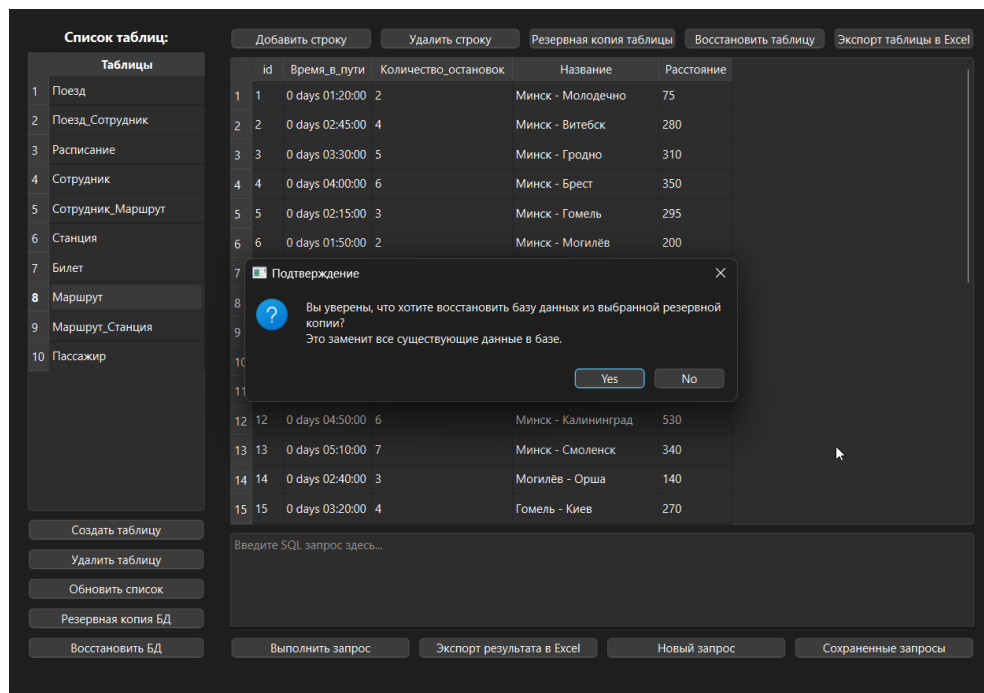


Рисунок 1.21 – Подтверждение восстановления базы данных

1.15 Кнопка «Восстановить таблицу»

Для того, чтобы восстановить таблицу из резервной копии необходимо кликнуть по кнопке «Восстановить таблицу». В новом окне выбрать путь к файлу с резервной копией таблицы. Подтвердить восстановление, кликнув по кнопке «Yes» (рисунок 1.22).

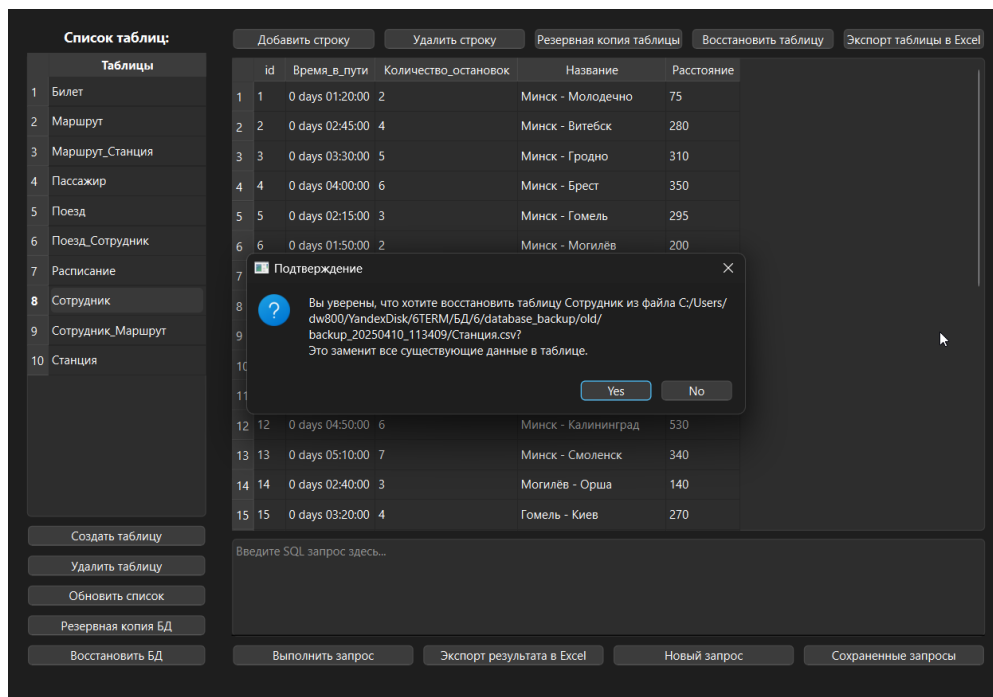


Рисунок 1.22 – Подтверждение восстановления таблицы

1.16 Изменение значений полей таблицы

Для того, чтобы изменить значение полей таблицы необходимо два раза кликнуть по значению, которое необходимо изменить, записать туда новое значение и кликнуть по кнопке «ОК».

На рисунках 1.23 и 1.24 представлены таблицы до и после изменения соответственно.

id	ФИО	Номер_паспорта	Пол	Контактный_телефон
18	Мельникова Юлия Андреевна	BB8234567	Женский	375298889900
19	Тимофеев Игорь Владимирович	BB9234567	Мужской	375299990011
20	Сорокина Надежда Сергеевна	CC1234567	Женский	375291112233
21	Борисов Аркадий Геннадьевич	CC2234567	Мужской	375292223344
22	Никитина Лариса Ивановна	CC3234567	Женский	375293334455
23	Савельев Дмитрий Викторович	CC4234567	Мужской	375294445566
24	Фролова Оксана Владимировна	CC5234567	Женский	375295556677
25	Рябов Евгений Анатольевич	CC6234567	Мужской	375296667788
26	Полякова Мария Олеговна	CC7234567	Женский	375297778899
27	Щербаков Алексей Павлович	CC8234567	Мужской	375298889900
28	Шевченко Галина Андреевна	CC9234567	Женский	375299990011
29	Федоров Николай Семенович	DD1234567	Мужской	375291112244
30	Емельянова Ольга Константиновна	DD2239567	Женский	375299997011
31	Сорокина Галина Андреевна	CC9664566	Женский	375288990011
32	Барсуков Николай Семенович	DD1114567	Мужской	375295512244

Рисунок 1.23 – Таблица Пассажиры до изменения

id	ФИО	Номер_паспорта	Пол	Контактный_телефон
18	Мельникова Юлия Андреевна	BB8234567	Женский	375298889900
19	Тимофеев Игорь Владимирович	BB9234567	Мужской	375299990011
20	Сорокина Надежда Сергеевна	CC1234567	Женский	375291112233
21	Борисов Аркадий Геннадьевич	CC2234567	Мужской	375292223344
22	Никитина Лариса Ивановна	CC3234567	Женский	375293334455
23	Савельев Дмитрий Викторович	CC4234567	Мужской	375294445566
24	Фролова Оксана Владимировна	CC5234567	Женский	375295556677
25	Рябов Евгений Анатольевич	CC6234567	Мужской	375296667788
26	Полякова Мария Олеговна	CC7234567	Женский	375297778899
27	Щербаков Алексей Павлович	CC8234567	Мужской	375298889900
28	Шевченко Галина Андреевна	CC9234567	Женский	375299990011
29	Федоров Николай Семенович	DD1234567	Мужской	375291112244
30	Емельянова Ольга Константиновна	DD2239567	Женский	375299997011
31	Сорокина Галина Андреевна	CC9664566	Женский	375288990011
32	Изменение значения ФИО	DD1114567	Мужской	375295512244

Ячейка успешно обновлена

Рисунок 1.24 – Таблица Пассажиры после изменения

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы было разработано настольное приложение для взаимодействия с базой данных, написанное на языке программирования Python 3.13 с использованием фреймворка Qt 6. Основное назначение программы – обеспечение удобного пользовательского интерфейса для выполнения операций с базой данных и визуализации результатов заданных запросов.

Реализована возможность создания, редактирования и удаления таблиц, и записей. Приложение поддерживает добавление новых полей, резервное копирование и восстановление как отдельных таблиц, так и всей базы данных.

Также внедрен модуль управления SQL-запросами, в котором предусмотрен механизм отображения заранее определенных запросов (в соответствии с лабораторными работами №4 и №5), а также возможность добавления новых пользовательских запросов и их сохранения. Результаты запросов можно экспортировать в файл для дальнейшего использования в формате Excel.

ПРИЛОЖЕНИЕ А
(обязательное)
Текст программы

```
import sys
import os
import datetime
import subprocess
import psycpg2
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget,
                              QVBoxLayout,
                              QHBoxLayout, QPushButton,
                              QTableWidgetItem,
                              QMessageBox, QInputDialog,
                              QFileDialog, QTextEdit,
                              QDialog, QLabel, QLineEdit,
                              QComboBox, QFormLayout)
from PyQt6.QtCore import Qt
import pandas as pd

class DatabaseManager:
    def __init__(self):
        self.conn = None
        self.cursor = None
        self.saved_queries = {}
        self.load_saved_queries()

    def load_saved_queries(self):
        try:
            with open('saved_queries.txt', 'r', encoding='utf-
8') as f:
                for line in f:
                    if ':' in line:
                        name, query = line.strip().split(':', 1)
                        self.saved_queries[name] = query
        except FileNotFoundError:
            pass

    def import_queries_from_file(self, file_path):
        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                for line in f:
                    if ':' in line:
                        name, query = line.strip().split(':', 1)
                        self.save_query(name, query)
            return True
        except Exception as e:
            QMessageBox.critical(None, 'Ошибка импорта
запросов', str(e))
            return False
```

```

def save_query(self, name, query):
    self.saved_queries[name] = query
    try:
        with open('saved_queries.txt', 'a', encoding='utf-
8') as f:
            f.write(f'{name}:{query}\n')
        return True
    except Exception as e:
        QMessageBox.critical(None, 'Ошибка сохранения
запроса', str(e))
        return False

def execute_query(self, query):
    try:
        # Экранируем имена таблиц, если они содержат только
цифры
        words = query.strip().split()
        for i, word in enumerate(words):
            # Удаляем возможные знаки пунктуации
            clean_word = word.rstrip(';').rstrip('')
            # Экранируем только имена таблиц/столбцов, но не
числовые значения в условиях
            if clean_word.isdigit() and i > 0 and words[i-
1].lower() not in ['where', 'and', 'or', '>', '<', '=', '>=',
'<=', '!=']:
                words[i] = words[i].replace(clean_word,
f'"{clean_word}"')
            query = ' '.join(words)

        # Проверяем, является ли запрос INSERT и есть ли
поле ID
        if query.upper().startswith('INSERT') and 'id' in
[col.lower() for col in self.cursor.description]:
            # Получаем информацию о типе поля ID
            table_name = query.split()[2] # Получаем имя
таблицы из запроса
            self.cursor.execute(f"""
                SELECT column_name, data_type,
column_default
                FROM information_schema.columns
                WHERE table_name = '{table_name}' AND
column_name = 'id'
            """)
            id_info = self.cursor.fetchone()

            # Если поле ID имеет тип SERIAL, не передаем
значение для него
            if id_info and 'nextval' in str(id_info[2]):
                # Модифицируем запрос, чтобы не передавать
значение для ID
                query = query.replace('id,',
''.replace('id)', ')).replace('VALUES (NULL,', 'VALUES (')

```

```

        self.cursor.execute(query)
        self.conn.commit()
        if self.cursor.description:
            columns = [desc[0] for desc in
self.cursor.description]
            data = self.cursor.fetchall()
            return True, columns, data
        return True, None, None
    except Exception as e:
        self.conn.rollback()
        return False, None, str(e)

    def create_new_table(self, table_name, columns):
        try:
            # Проверяем, что имя таблицы не содержит пробелов и
кириллицы
            # Экранируем имя таблицы
            escaped_table_name = f'"{table_name}"'

            # Добавляем поле ID с автоинкрементом, если его нет
в колонках
            has_id = any(col['name'].lower() == 'id' for col in
columns)
            if not has_id:
                columns.insert(0, {'name': 'id', 'type': 'SERIAL
PRIMARY KEY'})

            # Экранируем имена столбцов
            column_definitions = [f'"{col["name"]}"'
{col["type"]} ' for col in columns]
            create_query = f"CREATE TABLE {escaped_table_name}
({' , '.join(column_definitions)})"
            self.cursor.execute(create_query)

            # Добавляем пустую строку в новую таблицу (исключая
поле ID)
            column_names = [f'"{col["name"]}"' for col in
columns if col['name'].lower() != 'id']
            if column_names: # Проверяем, есть ли столбцы для
вставки
                placeholders = ' , '.join(['NULL'] *
len(column_names))
                insert_query = f"INSERT INTO
{escaped_table_name} ({' , '.join(column_names)}) VALUES
({placeholders})"
                self.cursor.execute(insert_query)

            self.conn.commit()
            return True
        except Exception as e:
            self.conn.rollback()

```

```

        QMessageBox.critical(None, 'Ошибка создания
таблицы', str(e))
        return False

    def delete_table(self, table_name):
        try:
            # Экранируем имя таблицы, если оно содержит только
цифры
            if table_name.isdigit():
                table_name = f'"{table_name}"'
            self.cursor.execute(f"DROP TABLE {table_name}")
            self.conn.commit()
            return True
        except Exception as e:
            self.conn.rollback()
            QMessageBox.critical(None, 'Ошибка удаления
таблицы', str(e))
            return False

    def export_to_excel(self, table_name):
        try:
            self.cursor.execute(f"SELECT * FROM {table_name}")
            data = self.cursor.fetchall()
            columns = [desc[0] for desc in
self.cursor.description]
            df = pd.DataFrame(data, columns=columns)

            file_path, _ = QFileDialog.getSaveFileName(
                None,
                'Сохранить как Excel',
                f'{table_name}.xlsx',
                'Excel Files (*.xlsx)'
            )

            if file_path:
                df.to_excel(file_path, index=False,
engine='openpyxl')
                return True
            return False
        except ImportError:
            QMessageBox.critical(None, 'Ошибка', 'Модуль
openpyxl не установлен. Установите его командой: pip install
openpyxl')
            return False
        except Exception as e:
            QMessageBox.critical(None, 'Ошибка экспорта',
str(e))
            return False

    def get_table_structure(self, table_name):
        try:

```

```

        # Экранируем имя таблицы, если оно содержит только
цифры
        if table_name.isdigit():
            table_name = f'"{table_name}"'
        self.cursor.execute(f"""
            SELECT column_name, data_type
            FROM information_schema.columns
            WHERE table_name = {table_name if
table_name.startswith('"') else f'"{table_name}"'}
            ORDER BY ordinal_position
        """)
        return self.cursor.fetchall()
    except Exception as e:
        QMessageBox.critical(None, 'Ошибка получения
структуры таблицы', str(e))
        return []

def connect(self):
    try:
        self.conn = psycopg2.connect(
            host='localhost',
            port=5432,
            database='Railway',
            user='postgres',
            password='1111'
        )
        self.cursor = self.conn.cursor()
        return True
    except Exception as e:
        QMessageBox.critical(None, 'Ошибка подключения',
str(e))
        return False

def disconnect(self):
    if self.cursor:
        self.cursor.close()
    if self.conn:
        self.conn.close()

def get_tables(self):
    self.cursor.execute("""
        SELECT table_name FROM information_schema.tables
        WHERE table_schema = 'public'
    """)
    return [table[0] for table in self.cursor.fetchall()]

def backup_table(self, table_name):
    try:
        # Получаем структуру таблицы
        self.cursor.execute(f"SELECT * FROM {table_name}
LIMIT 0")

```

```

        columns = [desc[0] for desc in
self.cursor.description]

        # Получаем данные
        self.cursor.execute(f"SELECT * FROM {table_name}")
        data = self.cursor.fetchall()

        # Сохраняем в DataFrame и экспортируем в CSV
        df = pd.DataFrame(data, columns=columns)
        backup_path = f'backup_{table_name}.csv'
        df.to_csv(backup_path, index=False)
        return True, backup_path
    except Exception as e:
        QMessageBox.critical(None, 'Ошибка резервного
копирования', str(e))
        return False, None

    def restore_table(self, backup_path):
        try:
            # Проверяем существование файла резервной копии
            if not os.path.exists(backup_path):
                raise FileNotFoundError(f"Файл резервной копии
{backup_path} не найден")

            # Загружаем данные из CSV
            df = pd.read_csv(backup_path)

            # Получаем имя таблицы из имени файла резервной
копии
            table_name =
os.path.splitext(os.path.basename(backup_path))[0]

            # Экранируем имя таблицы
            escaped_table_name = f'"{table_name}"'

            # Проверяем существование таблицы
            self.cursor.execute(f"""
                SELECT EXISTS (
                    SELECT FROM information_schema.tables
                    WHERE table_schema = 'public' AND table_name
= '{table_name}'
                );
            """)
            table_exists = self.cursor.fetchone()[0]

            if not table_exists:
                # Если таблица не существует, создаем ее
                columns = df.columns.tolist()

                # Проверяем наличие поля ID и его тип
                has_id = 'id' in [col.lower() for col in
columns]

```



```

        column_defs = []

        for col in columns:
            if col.lower() == 'id':
                # Если это поле ID, устанавливаем тип
                SERIAL PRIMARY KEY
                column_defs.append(f'"{col}" SERIAL
                PRIMARY KEY')
            else:
                # Автоматически определяем тип данных
                для столбца
                sample_value = df[col].dropna().iloc[0]
                if not df[col].dropna().empty else ''
                if isinstance(sample_value, (int,
                float)) or (isinstance(sample_value, str) and
                sample_value.replace('.', '', 1).isdigit()):
                    column_defs.append(f'"{col}"
                    NUMERIC')
                else:
                    column_defs.append(f'"{col}" TEXT')

            create_query = f"CREATE TABLE
            {escaped_table_name} ({', '.join(column_defs)});"
            self.cursor.execute(create_query)
        else:
            # Если таблица существует, очищаем ее с
            сохранением структуры
            self.cursor.execute(f"TRUNCATE TABLE
            {escaped_table_name}")

            # Вставляем данные из резервной копии
            if not df.empty:
                columns = df.columns.tolist()

                # Если есть поле ID, исключаем его из запроса
                INSERT
                if 'id' in [col.lower() for col in columns]:
                    columns = [col for col in columns if
                    col.lower() != 'id']

                if columns: # Проверяем, остались ли столбцы
                для вставки
                    placeholders = ', '.join(['%s'] *
                    len(columns))
                    query = f"INSERT INTO {escaped_table_name}
                    ({', '.join(columns)}) VALUES ({placeholders})"

                    for _, row in df.iterrows():
                        # Преобразуем строку DataFrame в список
                        значений, исключая ID если он есть
                        values = [None if pd.isna(val) else val

```

```

        for col, val in zip(df.columns,
row.tolist())
        if col.lower() != 'id']
        self.cursor.execute(query, values)

        self.conn.commit()
        return True
    except Exception as e:
        self.conn.rollback()
        QMessageBox.critical(None, 'Ошибка восстановления
таблицы', str(e))
        return False

    def backup_database(self):
        try:
            # Проверяем доступность pg_dump в PATH
            try:
                subprocess.run(['pg_dump', '--version'],
check=True, capture_output=True)
            except (subprocess.CalledProcessError,
FileNotFoundError):
                raise Exception("pg_dump не найден в PATH.
Убедитесь, что PostgreSQL установлен и pg_dump доступен")

            # Проверяем соединение с базой данных
            if not self.conn or self.conn.closed != 0:
                if not self.connect():
                    raise Exception("Не удалось подключиться к
базе данных")

            # Создаем директорию для резервных копий, если она
не существует
            backup_dir = 'database_backup'
            try:
                if not os.path.exists(backup_dir):
                    os.makedirs(backup_dir)
            except OSError as e:
                raise Exception(f"Не удалось создать директорию
для резервных копий: {str(e)}")

            # Текущая дата и время для имени файла
            timestamp =
datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
            backup_file = os.path.join(backup_dir,
f'backup_{timestamp}.sql')

            # Формируем команду для pg_dump с явным указанием
кодировки UTF-8
            dump_command = [
                'pg_dump',
                '-h', 'localhost',
                '-p', '5432',

```

```

        '-U', 'postgres',
        '-d', 'Railway',
        '--encoding=UTF8',
        '-F', 'p',
        '-f', backup_file
    ]

    # Создаем процесс с передачей пароля через
переменную окружения
    env = os.environ.copy()
    env['PGPASSWORD'] = '1111'
    env['PYTHONIOENCODING'] = 'utf-8'

    # Выполняем pg_dump без использования shell=True
    process = subprocess.Popen(
        dump_command,
        env=env,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        encoding='utf-8'
    )

    # Ждем завершения процесса
    stdout, stderr = process.communicate()

    if process.returncode != 0:
        error_msg = stderr if stderr else "Неизвестная
ошибка при выполнении pg_dump"
        raise Exception(f'Ошибка pg_dump: {error_msg}')

    # Проверяем, что файл резервной копии создан
    if not os.path.exists(backup_file) or
os.path.getsize(backup_file) == 0:
        raise Exception("Файл резервной копии не создан
или пуст")

    return True, backup_file
except Exception as e:
    QMessageBox.critical(None, 'Ошибка резервного
копирования базы данных', str(e))
    return False, None

def restore_database(self, backup_dir=None):
    try:
        # Если директория не указана, запрашиваем у
пользователя
        if backup_dir is None:
            backup_dir = QFileDialog.getExistingDirectory(
                None,
                'Выберите папку с резервной копией',
                'database_backup',
                QFileDialog.Option.ShowDirsOnly
            )

```

```

    )
    if not backup_dir:
        return False

    # Получаем список файлов .sql в директории
    backup_files = [f for f in os.listdir(backup_dir) if
f.endswith('.sql')]
    if not backup_files:
        QMessageBox.critical(None, 'Ошибка', 'В
указанной папке нет файлов резервных копий (.sql)')
        return False

    # Если несколько файлов, предлагаем выбрать один
    if len(backup_files) > 1:
        backup_file, ok = QInputDialog.getItem(
            None,
            'Выбор файла резервной копии',
            'Выберите файл для восстановления:',
            backup_files,
            0,
            False
        )
        if not ok or not backup_file:
            return False
    else:
        backup_file = backup_files[0]

    backup_file = os.path.join(backup_dir, backup_file)

    # Проверяем существование файла резервной копии
    if not os.path.exists(backup_file):
        raise FileNotFoundError(f"Файл резервной копии
{backup_file} не найден")

    # Проверяем права доступа к файлу
    if not os.access(backup_file, os.R_OK):
        try:
            subprocess.run(['icaccls', backup_file,
'/grant', '*S-1-1-0:(R)'], check=True)
        except subprocess.CalledProcessError as e:
            QMessageBox.warning(None, 'Предупреждение',
f"Не удалось установить права доступа к
файлу резервной копии: {str(e)}")
            return False

    # Проверяем кодировку файла
    try:
        with open(backup_file, 'rb') as f:
            f.read().decode('utf-8')
    except UnicodeDecodeError:
        QMessageBox.critical(None, 'Ошибка кодировки',
'Файл резервной копии имеет неверную кодировку (не UTF-8)')

```

```

        return False

    # Закрываем текущее соединение
    self.disconnect()

    # Пересоздаем базу данных
    temp_conn = psycopg2.connect(
        host='localhost',
        port=5432,
        database='postgres',  # Подключаемся к системной
БД
        user='postgres',
        password='1111'
    )
    temp_conn.autocommit = True
    temp_cursor = temp_conn.cursor()

    # Отключаем активные подключения к базе данных
    temp_cursor.execute("""
        SELECT
pg_terminate_backend(pg_stat_activity.pid)
        FROM pg_stat_activity
        WHERE pg_stat_activity.datname = 'Railway'
        AND pid <> pg_backend_pid()
    """)

    # Удаляем базу данных если она существует
    temp_cursor.execute("DROP DATABASE IF EXISTS
\"Railway\"")

    # Создаем новую базу данных
    temp_cursor.execute("CREATE DATABASE \"Railway\"")

    # Закрываем временное соединение
    temp_cursor.close()
    temp_conn.close()

    # Формируем команду для восстановления с явным
указанием кодировки
    restore_command = [
        'psql',
        '-h', 'localhost',
        '-p', '5432',
        '-U', 'postgres',
        '-d', 'Railway',
        '-f', backup_file,
        '-E', 'UTF8'
    ]

    # Создаем процесс с передачей пароля через
переменную окружения
    env = os.environ.copy()

```

```

env['PGPASSWORD'] = '1111'
env['PYTHONIOENCODING'] = 'utf-8'

# Выполняем восстановление
process = subprocess.Popen(
    restore_command,
    env=env,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    encoding='utf-8'
)

# Ждем завершения процесса
stdout, stderr = process.communicate()

if process.returncode != 0:
    raise Exception(f'Ошибка восстановления:
{stderr}')

# Переподключаемся к базе данных
return self.connect()
except Exception as e:
    QMessageBox.critical(None, 'Ошибка восстановления
базы данных', str(e))
    return False

def export_query_results_to_excel(self, query):
    try:
        # Выполняем запрос
        self.cursor.execute(query)
        data = self.cursor.fetchall()
        columns = [desc[0] for desc in
self.cursor.description]

        # Создаем DataFrame
        df = pd.DataFrame(data, columns=columns)

        # Запрашиваем путь для сохранения файла
        file_path, _ = QFileDialog.getSaveFileName(
            None,
            'Сохранить результаты запроса как Excel',
            'query_results.xlsx',
            'Excel Files (*.xlsx)'
        )

        if file_path:
            df.to_excel(file_path, index=False)
            return True
        return False
    except Exception as e:
        QMessageBox.critical(None, 'Ошибка экспорта
результатов запроса', str(e))

```

```

        return False

class QueryDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle('Новый запрос')
        self.setModal(True)

        layout = QVBoxLayout(self)

        layout.addWidget(QLabel('Название запроса:'))
        self.name_edit = QLineEdit()
        layout.addWidget(self.name_edit)

        layout.addWidget(QLabel('SQL запрос:'))
        self.query_edit = QTextEdit()
        layout.addWidget(self.query_edit)

        btn_layout = QHBoxLayout()
        save_btn = QPushButton('Сохранить')
        save_btn.clicked.connect(self.accept)
        import_btn = QPushButton('Добавить из файла')
        import_btn.clicked.connect(self.import_queries)
        cancel_btn = QPushButton('Отмена')
        cancel_btn.clicked.connect(self.reject)
        btn_layout.addWidget(save_btn)
        btn_layout.addWidget(import_btn)
        btn_layout.addWidget(cancel_btn)
        layout.addLayout(btn_layout)

    def import_queries(self):
        file_path, _ = QFileDialog.getOpenFileName(
            self,
            'Выберите файл с запросами',
            '',
            'Text Files (*.txt)'
        )
        if file_path:
            if
self.parent().db.import_queries_from_file(file_path):
                QMessageBox.information(self, 'Успех', 'Запросы
успешно импортированы')
                self.accept()

class AddRowDialog(QDialog):
    def __init__(self, parent=None, structure=None):
        super().__init__(parent)
        self.setWindowTitle('Добавление новой строки')
        self.setModal(True)
        self.structure = structure
        self.inputs = {}

```

```

        layout = QVBoxLayout(self)
        form_layout = QFormLayout()

        # Создаем поля ввода для каждого столбца таблицы
        for column_name, data_type in structure:
            # Если это поле id, показываем метку 'auto' вместо
поля ввода
            if column_name.lower() == 'id':
                label = QLabel('auto (заполнится
автоматически)')
                label.setStyleSheet('color: gray; font-style:
italic;')
                form_layout.addRow(f"{column_name}
({data_type}):", label)
                # Добавляем пустое поле в словарь, чтобы
сохранить порядок полей
                self.inputs[column_name] = None
            elif data_type.upper() in ['BOOLEAN']:
                input_field = QComboBox()
                input_field.addItems(['TRUE', 'FALSE'])
                form_layout.addRow(f"{column_name}
({data_type}):", input_field)
                self.inputs[column_name] = input_field
            else:
                input_field = QLineEdit()
                form_layout.addRow(f"{column_name}
({data_type}):", input_field)
                self.inputs[column_name] = input_field

        layout.addLayout(form_layout)

        btn_layout = QHBoxLayout()
        save_btn = QPushButton('Сохранить')
        save_btn.clicked.connect(self.accept)
        cancel_btn = QPushButton('Отмена')
        cancel_btn.clicked.connect(self.reject)
        btn_layout.addWidget(save_btn)
        btn_layout.addWidget(cancel_btn)
        layout.addLayout(btn_layout)

    def get_values(self):
        values = []
        columns = []
        for column_name, data_type in self.structure:
            # Пропускаем поле id, чтобы оно заполнялось
автоматически
            if column_name.lower() == 'id':
                continue

            input_field = self.inputs[column_name]

            if isinstance(input_field, QComboBox):

```



```

        value = input_field.currentText()
    elif input_field is not None:
        value = input_field.text()
        # Преобразуем пустые строки в None
        if value == '':
            value = None
    else:
        value = None

    values.append(value)
    columns.append(column_name)

return values, columns

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.db = DatabaseManager()
        self.init_ui()

        # Создаем строку состояния
        self.statusBar().showMessage('Готово к работе')

        if not self.db.connect():
            sys.exit(1)

        # Отображаем список таблиц при запуске
        self.update_tables_list()

    def init_ui(self):
        self.setWindowTitle('Управление базой данных Railway')
        self.setGeometry(100, 100, 1000, 700)

        # Создаем центральный виджет и главный layout
        central_widget = QWidget()
        self.setCentralWidget(central_widget)
        main_layout = QHBoxLayout(central_widget)

        # Создаем левую панель для списка таблиц
        left_panel = QWidget()
        left_layout = QVBoxLayout(left_panel)

        # Добавляем заголовок для списка таблиц
        tables_label = QLabel('Список таблиц:')
        tables_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        tables_label.setStyleSheet('font-weight: bold; font-
size: 14px;')
        left_layout.addWidget(tables_label)

        # Создаем список таблиц
        self.tables_list = QTableWidget()
        self.tables_list.setColumnCount(1)

```

```

        self.tables_list.setHorizontalHeaderLabels(['Таблицы'])

self.tables_list.horizontalHeader().setStretchLastSection(True)

self.tables_list.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)

self.tables_list.setSelectionMode(QTableWidget.SelectionMode.SingleSelection)

self.tables_list.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)

self.tables_list.itemClicked.connect(self.load_table_data)
    left_layout.addWidget(self.tables_list)

    # Кнопки для работы с таблицами
    btn_create_table = QPushButton('Создать таблицу')
    btn_delete_table = QPushButton('Удалить таблицу')
    btn_refresh_tables = QPushButton('Обновить список')

    # Добавляем кнопки на левую панель
    left_layout.addWidget(btn_create_table)
    left_layout.addWidget(btn_delete_table)
    left_layout.addWidget(btn_refresh_tables)

    # Добавляем кнопки для резервного копирования и
восстановления всей БД
    btn_backup_db = QPushButton('Резервная копия БД')
    btn_restore_db = QPushButton('Восстановить БД')
    left_layout.addWidget(btn_backup_db)
    left_layout.addWidget(btn_restore_db)

    # Устанавливаем фиксированную ширину для левой панели
    left_panel.setFixedWidth(200)
    main_layout.addWidget(left_panel)

    # Создаем правую панель для отображения данных и
запросов
    right_panel = QWidget()
    right_layout = QVBoxLayout(right_panel)

    # Создаем панель кнопок для работы с данными таблицы
    table_button_layout = QHBoxLayout()

    # Кнопки для работы с данными таблицы
    btn_add_row = QPushButton('Добавить строку')
    btn_delete_row = QPushButton('Удалить строку')
    btn_backup_table = QPushButton('Резервная копия
таблицы')
    btn_restore_table = QPushButton('Восстановить таблицу')

```

```

        btn_export_table = QPushButton('Экспорт таблицы в
Excel')

        # Добавляем кнопки на панель работы с таблицей
        table_button_layout.addWidget(btn_add_row)
        table_button_layout.addWidget(btn_delete_row)
        table_button_layout.addWidget(btn_backup_table)
        table_button_layout.addWidget(btn_restore_table)
        table_button_layout.addWidget(btn_export_table)

        # Создаем таблицу для отображения данных
        self.table_widget = QTableWidget()

self.table_widget.setSelectionBehavior(QTableWidget.SelectionBeh
avior.SelectRows)

self.table_widget.setSelectionMode(QTableWidget.SelectionMode.Si
ngleSelection)

self.table_widget.setEditTriggers(QTableWidget.EditTrigger.NoEdi
tTriggers)
        # Подключаем сигнал двойного клика по ячейке

self.table_widget.cellDoubleClicked.connect(self.cell_double_cli
cked)

        # Добавляем поле для ввода SQL запросов
        self.query_edit = QTextEdit()
        self.query_edit.setPlaceholderText('Введите SQL запрос
здесь...')
        self.query_edit.setMaximumHeight(100)

        # Создаем панель кнопок для работы с SQL запросами
        sql_button_layout = QHBoxLayout()

        # Кнопки для работы с SQL запросами
        btn_execute_query = QPushButton('Выполнить запрос')
        btn_export_query = QPushButton('Экспорт результата в
Excel')
        btn_new_query = QPushButton('Новый запрос')
        btn_run_saved_query = QPushButton('Сохраненные запросы')

        # Добавляем кнопки на панель работы с SQL
        sql_button_layout.addWidget(btn_execute_query)
        sql_button_layout.addWidget(btn_export_query)
        sql_button_layout.addWidget(btn_new_query)
        sql_button_layout.addWidget(btn_run_saved_query)

        # Добавляем все элементы в правый layout
        right_layout.addLayout(table_button_layout)
        right_layout.addWidget(self.table_widget)
        right_layout.addWidget(self.query_edit)

```

```

right_layout.addLayout(sql_button_layout)

# Добавляем правую панель в главный layout
main_layout.addWidget(right_panel)

# Подключаем сигналы к слотам
btn_create_table.clicked.connect(self.create_table)
btn_delete_table.clicked.connect(self.delete_table)

btn_refresh_tables.clicked.connect(self.update_tables_list)
btn_add_row.clicked.connect(self.add_row)
btn_delete_row.clicked.connect(self.delete_row)
btn_backup_table.clicked.connect(self.backup_table)
btn_restore_table.clicked.connect(self.restore_table)
btn_export_table.clicked.connect(self.export_to_excel)
btn_backup_db.clicked.connect(self.backup_database)
btn_restore_db.clicked.connect(self.restore_database)
btn_new_query.clicked.connect(self.new_query)

btn_run_saved_query.clicked.connect(self.run_saved_query)
btn_execute_query.clicked.connect(self.execute_query)

btn_export_query.clicked.connect(self.export_query_results)

def create_table(self):
    table_name, ok = QInputDialog.getText(
        self, 'Создание таблицы',
        'Введите имя таблицы:'
    )

    if not ok or not table_name:
        return

    columns = []
    while True:
        column_name, ok1 = QInputDialog.getText(
            self, 'Добавление столбца',
            'Введите имя столбца (или оставьте пустым для
завершения):'
        )

        if not ok1 or not column_name:
            break

        column_type, ok2 = QInputDialog.getItem(
            self, 'Тип данных',
            'Выберите тип данных:',
            ['INTEGER', 'TEXT', 'FLOAT', 'DATE', 'BOOLEAN'],
            0, False
        )

        if not ok2:

```

```

        break

        columns.append({'name': column_name, 'type':
column_type})

        if columns and self.db.create_new_table(table_name,
columns):
            QMessageBox.information(
                self, 'Успех',
                f'Таблица {table_name} успешно создана'
            )
            # Обновляем список таблиц
            self.update_tables_list()

def delete_table(self):
    # Проверяем, выбрана ли таблица в списке
    selected_items = self.tables_list.selectedItems()
    if not selected_items:
        QMessageBox.warning(self, 'Предупреждение',
'Выберите таблицу для удаления')
        return

    table_name = selected_items[0].text()
    if table_name:
        reply = QMessageBox.question(
            self, 'Подтверждение',
            f'Вы уверены, что хотите удалить таблицу
{table_name}?',
            QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No
        )

        if reply == QMessageBox.StandardButton.Yes:
            if self.db.delete_table(table_name):
                QMessageBox.information(
                    self, 'Успех',
                    f'Таблица {table_name} успешно удалена'
                )
                # Обновляем список таблиц
                self.update_tables_list()
                # Очищаем таблицу данных
                self.table_widget.setRowCount(0)
                self.table_widget.setColumnCount(0)
                # Удаляем ссылку на текущую таблицу
                if hasattr(self, 'current_table'):
                    delattr(self, 'current_table')

def backup_table(self):
    # Проверяем, выбрана ли таблица в списке
    selected_items = self.tables_list.selectedItems()
    if not selected_items:

```

```

        QMessageBox.warning(self, 'Предупреждение',
        'Выберите таблицу для резервного копирования')
        return

        table_name = selected_items[0].text()
        if table_name:
            success, backup_path =
self.db.backup_table(table_name)
            if success:
                QMessageBox.information(
                    self, 'Успех',
                    f'Резервная копия таблицы {table_name}
создана успешно: {backup_path}'
                )

    def restore_table(self):
        # Проверяем, выбрана ли таблица в списке
        selected_items = self.tables_list.selectedItems()
        if not selected_items:
            QMessageBox.warning(self, 'Предупреждение',
            'Выберите таблицу для восстановления')
            return

        table_name = selected_items[0].text()
        if table_name:
            # Запрашиваем файл резервной копии
            backup_path, _ = QFileDialog.getOpenFileName(
                self,
                'Выберите файл резервной копии',
                '',
                'CSV Files (*.csv)'
            )

            if backup_path:
                reply = QMessageBox.question(
                    self, 'Подтверждение',
                    f'Вы уверены, что хотите восстановить
таблицу {table_name} из файла {backup_path}? \n'
                    'Это заменит все существующие данные в
таблице.',
                    QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No
                )

                if reply == QMessageBox.StandardButton.Yes:
                    if self.db.restore_table(backup_path):
                        QMessageBox.information(
                            self, 'Успех',
                            f'Таблица {table_name} успешно
восстановлена'
                        )
                    # Обновляем отображение таблицы

```

```

        self.load_table_data(selected_items[0])

    def backup_database(self):
        success, backup_path = self.db.backup_database()
        if success:
            QMessageBox.information(
                self, 'Успех',
                f'Резервная копия базы данных создана успешно:
{backup_path}'
            )

    def restore_database(self):
        # Запрашиваем директорию с резервной копией
        backup_dir = QFileDialog.getExistingDirectory(
            self,
            'Выберите директорию с резервной копией базы данных'
        )

        if backup_dir:
            reply = QMessageBox.question(
                self, 'Подтверждение',
                'Вы уверены, что хотите восстановить базу данных
из выбранной резервной копии?\n'
                'Это заменит все существующие данные в базе.',
                QMessageBox.StandardButton.Yes |
                QMessageBox.StandardButton.No
            )

            if reply == QMessageBox.StandardButton.Yes:
                if self.db.restore_database(backup_dir):
                    QMessageBox.information(
                        self, 'Успех',
                        'База данных успешно восстановлена'
                    )
                    # Обновляем список таблиц
                    self.update_tables_list()

    def export_to_excel(self):
        # Проверяем, выбрана ли таблица в списке
        selected_items = self.tables_list.selectedItems()
        if not selected_items:
            QMessageBox.warning(self, 'Предупреждение',
                'Выберите таблицу для экспорта в Excel')
            return

        table_name = selected_items[0].text()
        if table_name:
            if self.db.export_to_excel(table_name):
                QMessageBox.information(
                    self, 'Успех',
                    f'Таблица {table_name} успешно
экспортирована в Excel'
                )

```

```

    )

    def export_query_results(self):
        # Проверяем, есть ли данные в таблице
        if self.table_widget.rowCount() == 0:
            QMessageBox.warning(self, 'Предупреждение', 'Нет
данных для экспорта')
            return

        # Получаем заголовки столбцов
        columns = []
        for j in range(self.table_widget.columnCount()):

columns.append(self.table_widget.horizontalHeaderItem(j).text())

        # Получаем данные из таблицы
        data = []
        for i in range(self.table_widget.rowCount()):
            row = []
            for j in range(self.table_widget.columnCount()):
                item = self.table_widget.item(i, j)
                row.append(item.text() if item else '')
            data.append(row)

        # Создаем DataFrame
        df = pd.DataFrame(data, columns=columns)

        # Запрашиваем путь для сохранения файла
        file_path, _ = QFileDialog.getSaveFileName(
            self,
            'Сохранить результаты как Excel',
            'query_results.xlsx',
            'Excel Files (*.xlsx)'
        )

        if file_path:
            try:
                df.to_excel(file_path, index=False)
                QMessageBox.information(self, 'Успех', 'Данные
успешно экспортированы в Excel')
            except Exception as e:
                QMessageBox.critical(self, 'Ошибка', f'Ошибка
экспорта данных: {str(e)}')

    def add_row(self):
        # Проверяем, загружена ли таблица
        if not hasattr(self, 'current_table'):
            QMessageBox.warning(self, 'Предупреждение', 'Сначала
выберите таблицу')
            return

        # Получаем структуру таблицы

```



```

        structure =
self.db.get_table_structure(self.current_table)
        if not structure:
            QMessageBox.warning(self, 'Ошибка', f'Не удалось
получить структуру таблицы {self.current_table}')
            return

        # Проверяем, является ли поле ID автоинкрементным
        is_auto_id = False
        for column in structure:
            if column[0].lower() == 'id':
                self.db.cursor.execute(f"""
                SELECT column_default
                FROM information_schema.columns
                WHERE table_name = '{self.current_table}'
AND column_name = 'id'
                """)
                id_info = self.db.cursor.fetchone()
                if id_info and 'nextval' in str(id_info[0]):
                    is_auto_id = True
                break

        # Создаем диалог для ввода данных
        dialog = AddRowDialog(self, structure)
        if dialog.exec():
            # Получаем введенные данные и названия столбцов
            values, columns = dialog.get_values()

            if not columns:
                QMessageBox.warning(self, 'Предупреждение', 'Нет
данных для вставки')
                return

            # Если ID автоинкрементный, исключаем его из запроса
            if is_auto_id:
                # Сохраняем оригинальные columns и values
                orig_columns = columns
                orig_values = values

                # Фильтруем только не-ID колонки
                columns = [col for col in columns if col.lower()
!= 'id']

                values = [val for col, val in zip(orig_columns,
orig_values) if col.lower() != 'id']

                # Если остались другие колонки кроме ID,
продолжаем вставку
                if columns:
                    pass
                # Если остался только ID, но есть другие
значения - все равно вставляем

```

```

        elif len(orig_columns) > 1 or (len(orig_columns)
== 1 and len(orig_values) > 0):
            columns = [col for col in orig_columns if
col.lower() != 'id']
            values = [val for col, val in
zip(orig_columns, orig_values) if col.lower() != 'id']
            else:
                QMessageBox.warning(self, 'Предупреждение',
'Нет данных для вставки')
                return

        # Экранируем имя таблицы, если оно содержит только
цифры
        table_name = f'"{self.current_table}"' if
self.current_table.isdigit() else self.current_table

        # Создаем placeholders только для отфильтрованных
столбцов
        placeholders = ', '.join(['%s'] * len(columns))
        query = f"INSERT INTO {table_name} ({',
'.join(columns)}) VALUES ({placeholders})"

        # Выполняем запрос
        try:
            self.db.cursor.execute(query, values)
            self.db.conn.commit()
            QMessageBox.information(self, 'Успех', 'Строка
успешно добавлена')

            # Обновляем отображение таблицы
            selected_items =
self.tables_list.selectedItems()
            if selected_items:
                self.load_table_data(selected_items[0])
        except Exception as e:
            self.db.conn.rollback()
            QMessageBox.critical(self, 'Ошибка', f'Ошибка
добавления строки: {str(e)}')

    def new_query(self):
        dialog = QueryDialog(self)
        if dialog.exec():
            name = dialog.name_edit.text().strip()
            query = dialog.query_edit.toPlainText().strip()

            if name and query:
                if self.db.save_query(name, query):
                    QMessageBox.information(self, 'Успех',
'Запрос успешно сохранен')

    def execute_query(self):
        query = self.query_edit.toPlainText().strip()

```

```

        if not query:
            QMessageBox.warning(self, 'Предупреждение', 'Введите
SQL запрос')
            return

        success, columns, data = self.db.execute_query(query)

        if success:
            if columns and data:
                self.table_widget.setRowCount(len(data))
                self.table_widget.setColumnCount(len(columns))

self.table_widget.setHorizontalHeaderLabels(columns)

                for i, row in enumerate(data):
                    for j, value in enumerate(row):
                        item = QTableWidgetItem(str(value))
                        self.table_widget.setItem(i, j, item)

                # Настраиваем автоматическую подгонку ширины
столбцов под содержимое
                self.table_widget.resizeColumnsToContents()

                self.statusBar().showMessage('Запрос успешно
выполнен', 3000)
            else:
                self.statusBar().showMessage('Запрос успешно
выполнен (без данных для отображения)', 3000)
            else:
                QMessageBox.critical(self, 'Ошибка', f'Ошибка
выполнения запроса: {data}')

    def load_table_data(self, item):
        # Получаем имя выбранной таблицы
        table_name = item.text()

        # Экранируем имя таблицы, если оно содержит только цифры
        if table_name.isdigit():
            table_name = f'"{table_name}"'

        # Выполняем запрос для получения данных таблицы
        success, columns, data = self.db.execute_query(f"SELECT
* FROM {table_name}")

        if success and columns and data:
            self.table_widget.setRowCount(len(data))
            self.table_widget.setColumnCount(len(columns))
            self.table_widget.setHorizontalHeaderLabels(columns)

            for i, row in enumerate(data):
                for j, value in enumerate(row):
                    item = QTableWidgetItem(str(value))

```

```

        self.table_widget.setItem(i, j, item)

        # Настраиваем автоматическую подгонку ширины
        столбцов под содержимое
        self.table_widget.resizeColumnsToContents()

        # Сохраняем имя текущей таблицы
        self.current_table = table_name
        self.statusBar().showMessage(f'Загружена таблица:
{table_name}', 3000)
    else:
        QMessageBox.warning(self, 'Ошибка', f'Не удалось
загрузить данные таблицы {table_name}')

    def cell_double_clicked(self, row, column):
        # Проверяем, загружена ли таблица
        if not hasattr(self, 'current_table'):
            return

        # Получаем структуру таблицы
        structure =
self.db.get_table_structure(self.current_table)
        if not structure:
            return

        # Получаем имя столбца и его тип
        column_name =
self.table_widget.horizontalHeaderItem(column).text()
        column_type = None

        for col_name, col_type in structure:
            if col_name == column_name:
                column_type = col_type
                break

        # Не позволяем редактировать столбец id
        if column_name.lower() == 'id':
            QMessageBox.information(self, 'Информация', 'Столбец
ID не может быть отредактирован')
            return

        # Получаем текущее значение ячейки
        current_value = self.table_widget.item(row,
column).text()

        # Получаем значение первичного ключа (предполагаем, что
это первый столбец)
        id_column =
self.table_widget.horizontalHeaderItem(0).text()
        id_value = self.table_widget.item(row, 0).text()

        # Запрашиваем новое значение

```

```

        if column_type and column_type.upper() in ['BOOLEAN']:
            new_value, ok = QInputDialog.getItem(
                self, 'Редактирование ячейки',
                f'Введите новое значение для {column_name}:',
                ['TRUE', 'FALSE'], 0 if current_value.upper() ==
'TRUE' else 1, False
            )
        else:
            new_value, ok = QInputDialog.getText(
                self, 'Редактирование ячейки',
                f'Введите новое значение для {column_name}:',
                text=current_value
            )

        if ok and new_value != current_value:
            # Формируем SQL запрос для обновления
            query = f"UPDATE {self.current_table} SET
{column_name}='{new_value}' WHERE {id_column}='{id_value}'"

            # Выполняем запрос
            success, _, error = self.db.execute_query(query)

            if success:
                # Обновляем ячейку в таблице
                self.table_widget.item(row,
column).setText(new_value)
                self.statusBar().showMessage('Ячейка успешно
обновлена', 3000)
            else:
                QMessageBox.critical(self, 'Ошибка', f'Ошибка
обновления ячейки: {error}')

    def delete_row(self):
        # Проверяем, выбрана ли строка
        selected_rows =
self.table_widget.selectionModel().selectedRows()
        if not selected_rows:
            QMessageBox.warning(self, 'Предупреждение',
'Выберите строку для удаления')
            return

        # Проверяем, загружена ли таблица
        if not hasattr(self, 'current_table'):
            QMessageBox.warning(self, 'Предупреждение', 'Сначала
выберите таблицу')
            return

        # Получаем индекс выбранной строки
        row_index = selected_rows[0].row()

        # Получаем структуру таблицы для определения первичного
ключа

```

```

        structure =
self.db.get_table_structure(self.current_table)

        # Предполагаем, что первый столбец - это первичный ключ
или уникальный идентификатор
        id_column =
self.table_widget.horizontalHeaderItem(0).text()
        id_value = self.table_widget.item(row_index, 0).text()

        # Запрашиваем подтверждение
        reply = QMessageBox.question(
            self, 'Подтверждение',
            f'Вы уверены, что хотите удалить строку с
{id_column}={id_value}?',
            QMessageBox.StandardButton.Yes |
            QMessageBox.StandardButton.No
        )

        if reply == QMessageBox.StandardButton.Yes:
            # Выполняем запрос на удаление
            success, _, error = self.db.execute_query(
                f"DELETE FROM {self.current_table} WHERE
{id_column}='{id_value}'"
            )

            if success:
                # Обновляем отображение таблицы
                self.table_widget.removeRow(row_index)
                self.statusBar().showMessage('Строка успешно
удалена', 3000)
            else:
                QMessageBox.critical(self, 'Ошибка', f'Ошибка
удаления строки: {error}')

    def run_saved_query(self):
        if not self.db.saved_queries:
            QMessageBox.warning(self, 'Предупреждение', 'Нет
сохраненных запросов')
            return

        # Создаем диалоговое окно
        dialog = QDialog(self)
        dialog.setWindowTitle('Сохраненные запросы')
        layout = QVBoxLayout(dialog)

        # Создаем выпадающий список запросов
        combo = QComboBox()
        combo.addItem(list(self.db.saved_queries.keys()))
        layout.addWidget(combo)

        # Создаем кнопки

```

```

btn_layout = QHBoxLayout()
select_btn = QPushButton('Выбрать')
delete_btn = QPushButton('Удалить все запросы')
cancel_btn = QPushButton('Отмена')

btn_layout.addWidget(select_btn)
btn_layout.addWidget(delete_btn)
btn_layout.addWidget(cancel_btn)
layout.addLayout(btn_layout)

# Подключаем обработчик для кнопки удаления
delete_btn.clicked.connect(lambda:
self.delete_saved_queries(dialog))

# Подключаем обработчики событий
select_btn.clicked.connect(dialog.accept)
cancel_btn.clicked.connect(dialog.reject)

# Инициализируем переменную query_name
query_name = None

if dialog.exec():
    query_name = combo.currentText()

    if query_name:
        query = self.db.saved_queries[query_name]
        success, columns, data =
self.db.execute_query(query)

        if success:
            if columns and data:
                self.table_widget.setRowCount(len(data))

self.table_widget.setColumnCount(len(columns))

self.table_widget.setHorizontalHeaderLabels(columns)

        for i, row in enumerate(data):
            for j, value in enumerate(row):
                item =
QTableWidgetItem(str(value))
                self.table_widget.setItem(i, j,
item)

        # Настраиваем автоматическую подгонку
ширины столбцов под содержимое

self.table_widget.resizeColumnsToContents()

        QMessageBox.information(self, 'Успех',
'Запрос успешно выполнен')
        else:

```

```

        QMessageBox.information(self, 'Успех',
        'Запрос успешно выполнен (без данных для отображения)')
    else:
        QMessageBox.critical(self, 'Ошибка',
        f'Ошибка выполнения запроса: {data}')

    def delete_saved_queries(self, dialog):
        reply = QMessageBox.question(
            self, 'Подтверждение',
            'Вы уверены, что хотите удалить все сохраненные
запросы?',
            QMessageBox.StandardButton.Yes |
            QMessageBox.StandardButton.No
        )

        if reply == QMessageBox.StandardButton.Yes:
            try:
                # Очищаем словарь сохраненных запросов
                self.db.saved_queries.clear()
                # Очищаем файл с сохраненными запросами
                with open('saved_queries.txt', 'w',
encoding='utf-8') as f:
                    f.write('')
                QMessageBox.information(self, 'Успех', 'Все
сохраненные запросы удалены')
                dialog.reject() # Закрываем диалог после
удаления

            except Exception as e:
                QMessageBox.critical(self, 'Ошибка', f'Ошибка
удаления запросов: {str(e)}')

    def closeEvent(self, event):
        self.db.disconnect()
        event.accept()

    def update_tables_list(self):
        tables = self.db.get_tables()
        self.tables_list.setRowCount(len(tables))

        for i, table_name in enumerate(tables):
            item = QTableWidgetItem(table_name)
            self.tables_list.setItem(i, 0, item)

        if not tables:
            self.statusBar().showMessage('Нет доступных таблиц в
базе данных', 3000)

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

```